

Bases de datos no relacionales

[Descargar estos apuntes](#)

Índice

▼ Bases de datos no relacionales

- Introducción
- Ventajas de los sistemas noSQL
- Diferencias con las Bases de Datos SQL

▼ Tipos de Bases de Datos noSQL

- Clave-Valor
- Documentales
- Grafos
- Orientadas a objetos

▼ Algunos ejemplos de Bases de Datos noSQL

- Cassandra
- MongoDB
- CouchDB

- Usos y aplicaciones

▼ MongoDB

- Instalación

▼ Entorno de trabajo en MongoDB

- MongoDB Compass
- MongoDB Shell (mongosh)

- Bases de datos y colecciones en MongoDB

- Estructura de un documento MongoDB

▼ Tipos de datos

- Tipos primitivos
- Tipos complejos
- Tipos específicos (especializados)

- Operadores en MongoDB

▼ Gestión de bases de datos y colecciones

- Comandos relacionados con bases de datos
- Comandos relacionados con colecciones

▼ Consulta de documentos en colecciones

- Consultas directas
- Consultas de agregación (Aggregation Framework)
- ▼ Actualización de documentos en colecciones
 - Insertar documentos
 - Modificar documentos
 - Eliminar documentos
- Uso básico de MongoDB Compass
- ▼ Modelado de datos en MongoDB
 - Relaciones 1-1
 - Relaciones 1-N
 - Relaciones M-N
- Consultas multitaslas
- ▼ Otros aspectos importantes de MongoDB
 - Índices
 - Usuarios y roles
 - Scripts de MongoDB

Introducción

Hasta la fecha, la mayoría de los desarrolladores utilizaban sistemas de bases de datos SQL, pero de un tiempo a esta parte, un nuevo sistema de bases de datos parece haber llegado para quedarse, estos son los **sistemas de bases de datos NoSQL** (Not Only SQL – No sólo SQL).

Se puede decir que el término NoSQL aparece con la llegada de la Web 2.0. Hasta ese momento, sólo las grandes empresas subían contenidos a Internet si disponían de un portal web. Pero la aparición de plataformas como Facebook, Twitter, etc, cualquier usuario podía subir contenidos a Internet, lo que provocó un gran incremento de datos.

Es en este momento cuando comienzan a aparecer los primeros problemas en los sistemas de bases de datos relacionales. La primera opción fue ampliar máquinas, pero esta solución no funcionaba y resultaba muy costoso. La segunda opción fue crear un sistema específico a tal problema, obteniendo así una solución robusta al problema, de ahí surgen los sistemas de bases de datos NoSQL.

El uso de los sistemas de bases NoSQL permite superar los problemas de escalabilidad y rendimiento que se producen en los sistemas de bases de datos relacionales cuando se dan cita una gran cantidad de usuarios concurrentes y millones de consultas diarias. Estos sistemas de bases de datos NoSQL están optimizados para las tareas de recuperación y agregación de información.

Además, las bases de datos NoSQL son sistemas de almacenamiento de información que no cumplen el esquema Entidad-Relación, ni la estructura de tabla en la que se almacenan los datos. Estos

sistemas almacenan la información haciendo uso de formatos como **Clave-Valor**, **mapeo de columnas** o **grafos**.

Ventajas de los sistemas noSQL

La forma de almacenar de los sistemas de bases de datos NoSQL proporcionan una serie de ventajas sobre los sistemas de bases de datos relacionales.

- **Se pueden ejecutar sobre máquinas con pocos recursos:** estos sistemas, a diferencia de los sistemas basados en SQL, no requieren apenas computación, permitiendo así ejecutarse sobre máquinas de pocos recursos.
- **Escalabilidad horizontal:** para mejorar el rendimiento del sistema, únicamente es necesario añadir más nodos e indicar al sistema que nodos están disponibles.
- **Manejan grandes cantidades de datos:** esto es debido a que utilizan una estructura distribuida. En muchos casos se utilizan tablas Hash.
- **No generan cuellos de botella:** el principal problema de los sistemas SQL es que necesitan transcribir cada una de las sentencias para poder ejecutarlas. Esto significa que hay un punto de entrada común y, ante muchas peticiones puede ralentizar el sistema.
- **Existen diferentes sistemas de bases de datos NoSQL** según la naturaleza de los proyectos, esto permite elegir la base de datos más óptima para cada necesidad.

Diferencias con las Bases de Datos SQL

Estas son algunas de las diferencias más destacables entre los sistemas de bases de datos NoSQL y los sistemas SQL.

- **No utilizan SQL como lenguaje de consultas:** la mayor parte de los sistemas de bases de datos NoSQL no utilizan este lenguaje o simplemente como apoyo. Por ejemplo, MongoDB utiliza JSON, BigTable utiliza GQL y Cassandra utiliza CQL.
- **No utilizan estructuras de almacenamiento fijas para guardar la información:** hacen uso de otros modelos para el almacenamiento de la información, como sistemas de Clave-Valor, grafos u objetos.
- **No suelen permitir operaciones JOIN:** al tener un volumen de datos extremadamente grande, resulta incómodo utilizar acciones con JOIN ya que la sobrecarga en el sistema puede resultar muy costosa. La solución a este problema puede estar en la desnormalización de los datos o realizar JOIN por software en la capa de aplicación.
- **Arquitectura distribuida:** las bases de datos relacionales suelen estar centralizadas en una sola máquina o con estructura Cliente-Servidor. Pero los sistemas de bases de datos NoSQL pueden contener la información compartida entre varias máquinas mediante tablas Hash distribuidas.

Tipos de Bases de Datos noSQL

Según la forma que utilice de almacenar la información nos encontraremos con diferentes tipos de sistemas de bases de datos NoSQL, las más habituales son las que se muestran a continuación.

Clave-Valor

Estas son las habituales en los sistemas de bases de datos NoSQL ya que son las más sencillas en cuanto a funcionalidad se refiere. En este sistema, cada elemento es identificado por una clave única, permitiendo así una recuperación rápida. Además, la información es almacenada como un objeto binario (BLOB). Son muy eficientes en cuanto a lectura y escritura. Algunos ejemplos de este tipo son Cassandra, BigTable o Hbase.

Documentales

En el modelo documental la información se almacena como si de un documento se tratase, para ello utiliza una estructura simple como **JSON** o **XML**, en el cual se utilizará una clave única para cada registro. Además de realizar búsquedas por clave-valor, se pueden hacer búsquedas más avanzadas por el contenido del documento.

Este tipo de modelo es el más versátil, se puede utilizar en gran variedad de proyectos, incluyendo muchos que funcionarían con el modelo relacional. Algunos ejemplos serían **MongoDB** y **CouchDB**.

```
{
  Nombre:"Alberto",
  Dirección:"Castaños 17",
  Hijos:[
    {Nombre:"Andrés",   Edad:12},
    {Nombre:"Susana",   Edad:7},
    {Nombre:"Verónica", Edad:4}
  ]
}
```

Grafos

En este modelo la información se representa como nodos de un grafos y sus relaciones como las aristas entre los nodos, de forma que para recorrer la información se puede aplicar la teoría de grafos. Para conseguir un rendimiento máximo la información debe estar normalizada.

Este modelo permite una navegación más eficiente entre relaciones que el modelo relacional. Algunos ejemplos son **Neo4j**, **InfoGrid** y **Virtuoso**.

Orientadas a objetos

Este modelo representa la información mediante objetos, al igual que ocurre en la programación orientada a objetos (POO) con JAVA, C# o Visual Basic .Net. Algunos ejemplos de este modelo son Zope, Gemstone, Matisse y Db4o.



Más información de tipos de Bases de Datos noSQL

- [Bases de datos clave-valor](#)
- [Base de datos columnares](#)
- [Base de datos orientada a documentos](#)
- [Base de datos orientada a grafos](#)

Algunos ejemplos de Bases de Datos noSQL

A continuación algunos ejemplos de sistemas de bases de datos NoSQL más utilizadas actualmente.

Cassandra

Base de datos creada por Apache de tipo Clave-Valor. Dispone de su propio lenguaje de consultas CQL (Cassandra Query Language). Es una aplicación JAVA y puede correr sobre cualquier plataforma que disponga de JVM.

Puedes visitar <http://cassandra.apache.org/> para más información.



MongoDB

Esta base de datos creada por 10gen Inc íntegramente en C++ proporciona gran velocidad a la hora de ejecutar sus operaciones. Es de esquema libre orientada a documento, lo que permite que entrada sea diferente del resto de registros almacenados.

MongoDB utiliza un lenguaje propio para el almacenamiento de la información, BSON que es una evolución de JSON, con la particularidad de poder almacenar datos binarios. Actualmente, MongoDB es la base de datos preferida entre los desarrolladores.

Puedes visitar <https://www.mongodb.com/es> para más información.



CouchDB

Creada por Apache y desarrollada utilizando el lenguaje de programación Erlang4 es capaz de funcionar sobre la mayoría de sistemas operativos. Destacar que utiliza como lenguaje de interacción JavaScript y JSON para el almacenamiento de archivos.

Permite la creación de vistas, lo que permite combinar documentos para obtener valores, es decir, permite realizar operaciones JOIN típicas de los sistemas de bases de datos relacionales.



Puedes visitar <http://couchdb.apache.org/> para más información.

Más ejemplos de BD noSQL se pueden encontrar en <http://nosql-database.org/>.

Usos y aplicaciones

Algunas empresas que utilizan este tipo de bases de datos son las siguientes:

- **Cassandra:** Twitter, Facebook...
- **MongoDB:** FourSquare, SourceForge...
- **HBase:** Yahoo, Adobe...
- **Redis:** Flickr, Instagram, Github...
- **Neo4j:** Infojobs...
- **CouchDB:** BBC, The New York Times...

A continuación se dan unas directrices sobre cuándo elegir sistemas de bases de datos NoSQL:

- Cuando la escalabilidad en un modelo relacional no es viable, tanto a nivel técnico como de costes.
- Cuando el sistema tiene picos de uso muy elevados por parte de los usuarios.
- Cuando el número de datos crece rápidamente en situaciones puntuales, llegando a superar el Terabyte de información.
- Cuando la información no es homogénea, es decir, cuando en cada inserción podemos tener diferentes campos.

Ranking de Bases de Datos más usadas

Información	Tipo
https://db-engines.com/en/ranking	General
https://db-engines.com/en/ranking/relational+dbms	SQL Relacionales

Ranking de Bases de Datos noSQL más usadas

Información	Tipo
https://db-engines.com/en/ranking/document+store	BD Documentales
https://db-engines.com/en/ranking/key-value+store	BD Clave-Valor
https://db-engines.com/en/ranking/native+xml+dbms	BD XML
https://db-engines.com/en/ranking/object+oriented+dbms	BD Orientada Objetos
https://db-engines.com/en/ranking/graph+dbms	BD Grafos

MongoDB

MongoDB es un sistema de gestión de bases de datos NoSQL de tipo **document-oriented**, cuya unidad fundamental de almacenamiento es el **documento**. Estos documentos constituyen estructuras semiestructuradas basadas en **pares clave–valor**, lo que permite representar información de manera flexible y expresiva. Internamente, MongoDB utiliza **BSON (Binary JSON)**, un formato binario que extiende las capacidades del JSON tradicional al incorporar nuevos tipos de datos, soporte nativo para información binaria y mecanismos optimizados de lectura y escritura. Gracias a ello, MongoDB combina la simplicidad conceptual del JSON con un rendimiento propio de sistemas altamente optimizados.

El **modelo documental** de MongoDB se caracteriza por varias propiedades esenciales. En primer lugar, destaca su **flexibilidad estructural**, ya que no requiere un esquema rígido previo; cada documento puede evolucionar de manera independiente sin afectar al resto de la colección. En segundo lugar, posee una naturaleza **autodescriptiva**, dado que cada documento contiene tanto su estructura como los tipos de datos asociados a cada campo. Asimismo, se trata de un modelo **altamente expresivo**, capaz de representar relaciones complejas mediante documentos anidados y *arrays* heterogéneos. A ello se suma su **compatibilidad natural con JSON**, que facilita la integración con aplicaciones modernas, especialmente aquellas desarrolladas para entornos web y arquitecturas distribuidas.

MongoDB es, además, una base de datos **distribuida**, **general-purpose** y diseñada específicamente para responder a las necesidades de las aplicaciones contemporáneas y de la era de la nube. Su filosofía de diseño pretende maximizar la productividad del desarrollador, ofreciendo una experiencia de uso intuitiva y herramientas que permiten construir aplicaciones escalables sin complejidad innecesaria.

Como base de datos documental, MongoDB almacena los datos en forma de documentos de estilo JSON, lo que se alinea de manera natural con la lógica de las aplicaciones modernas y resulta conceptualmente más expresivo que el tradicional modelo relacional basado en filas y columnas. Esta representación facilita una comprensión más directa de la estructura de los datos y reduce la necesidad de transformaciones entre el modelo de aplicación y el modelo de persistencia.

El **lenguaje de consulta** de MongoDB es rico y expresivo: permite filtrar, ordenar y proyectar información en función de cualquier campo, sin importar el nivel de anidamiento dentro del documento. Además, el sistema soporta potentes mecanismos de **agregación**, que permiten llevar a cabo operaciones complejas como agrupaciones, transformaciones o cálculos derivados. MongoDB incorpora también capacidades avanzadas para casos de uso contemporáneos, como **búsqueda de texto**, **consultas geoespaciales**, **análisis de grafos** básicos y **procesamiento de grandes volúmenes de datos**. Una característica distintiva es que las propias consultas se expresan en formato JSON, lo que simplifica enormemente su construcción y evita recurrir a técnicas proclives a errores, como la concatenación de cadenas necesaria en SQL dinámico.

En conjunto, MongoDB ofrece un entorno moderno, flexible y especialmente adecuado para aplicaciones que requieren gran adaptabilidad, estructuración natural de la información y una experiencia de desarrollo fluida. Su modelo documental, basado en BSON, proporciona una base sólida para representar datos complejos de manera eficiente, al tiempo que dota al desarrollador de un lenguaje de consulta intuitivo y potente, coherente con la forma en que los datos se estructuran en la mayoría de aplicaciones actuales.

MongoDB integra una serie de características que definen su funcionamiento y su ecosistema de herramientas. En cuanto al **sistema gestor de bases de datos (SGBD)**, se trata del propio *MongoDB*, una plataforma diseñada para ofrecer un almacenamiento flexible y orientado a documentos. Su **modelo de datos** es de tipo documental, lo que significa que la información se organiza en documentos con estructura similar a objetos JSON, permitiendo representar relaciones complejas y estructuras anidadas de manera natural.

Los **tipos de datos** utilizados se basan en documentos JSON —o, más precisamente, en BSON a nivel interno— lo que favorece la compatibilidad con lenguajes de programación modernos y facilita la manipulación de datos en aplicaciones distribuidas. Para interactuar con el servidor, MongoDB emplea un **modelo cliente/servidor**, de modo que múltiples clientes pueden conectarse de manera

simultánea a través de la red. Dichas conexiones utilizan por defecto el **puerto 27017**, que actúa como canal principal de comunicación del servicio.

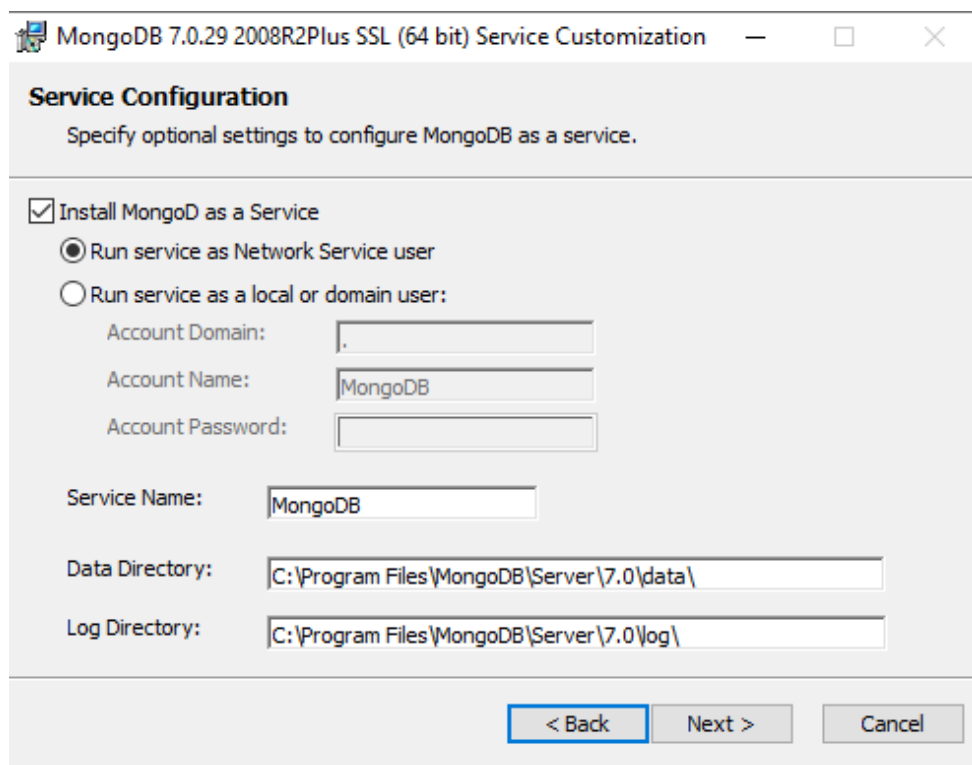
El ecosistema de MongoDB incluye diversas herramientas de acceso según las necesidades del usuario. La interfaz más directa es **mongosh**, el cliente de línea de comandos que permite ejecutar consultas y administrar la base de datos con precisión. Para quienes prefieren una experiencia visual, MongoDB ofrece **MongoDB Compass**, una aplicación gráfica de escritorio que facilita la exploración y análisis de colecciones. Como alternativa basada en navegador web, existe **MongoAdmin**, una interfaz gráfica que proporciona una forma ligera y accesible de gestionar la base de datos sin necesidad de instalar software adicional.

Estas características en conjunto proporcionan un entorno de trabajo versátil y adaptable, pensado tanto para desarrolladores que necesitan control detallado mediante línea de comandos como para quienes requieren herramientas visuales orientadas a la productividad.

Instalación

La instalación de MongoDB es bastante sencilla, ya que los manuales facilitados en su propia web hacen que esta tarea no sea complicada.

Desde la sección de descargas nos bajaremos la versión estable actual para Community Server en la versión de nuestro sistema operativo. En la instalación seleccionaremos la opción de **Completa** y luego lo configuraremos como servicio. La siguiente imagen muestra la pantalla donde seleccionaremos esta opción para que MongoDB se instale como servicio de Windows.



Después, dejaremos marcado **MongoDB Compass** para que se instale también junto con **MongoDB Shell**.

Para más información sobre la instalación, puedes consultar los siguientes enlaces según tu sistema operativo:



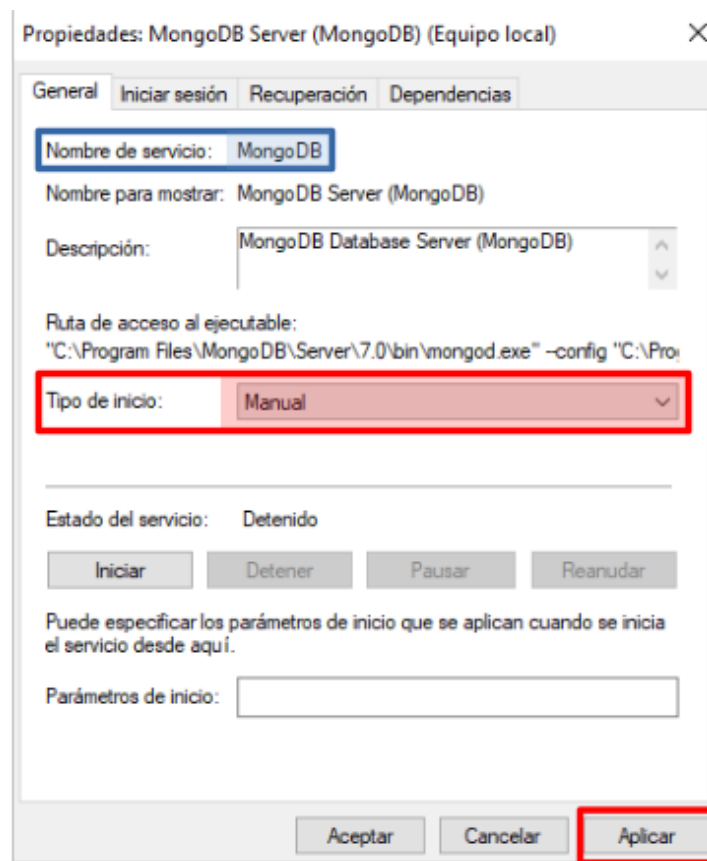
Referencias de instalación

- [Instalación para Linux](#)
- [Instalación para Windows](#)
- [Instalación para OS X](#)
- [Instalación para Linux Community Server](#)

Una vez finalizada la instalación, comprobaremos que el servicio se ha instalado correctamente desde el Administrador de Servicios de Windows (`services.msc`):

Podemos comprobar que el servicio está instalado correctamente, que por defecto se está ejecutando y que su **tipo de inicio** es **Automático**. También podemos ver su descripción y que el **ejecutable** se llama `mongod.exe`.

Es aconsejable dejar el servicio en Manual para que no se inicie automáticamente cada vez que iniciemos Windows. Una vez cambiado el tipo de inicio a Manual, deberemos clicar en el botón **Aplicar** para guardar los cambios.



El servicio podemos iniciarlo o detenerlo cuando lo necesitemos desde el Administrador de Servicios o desde la consola CMD como administrador con el comando `net` .

Desde la línea de comandos usaremos el comando `net start` para iniciar el servicio:

```
C:\Windows\system32>net start MongoDB
El servicio de MongoDB Server (MongoDB) está iniciándose.
El servicio de MongoDB Server (MongoDB) se ha iniciado correctamente.
```

Para detenerlo utilizaremos el comando `net stop` :

```
C:\Windows\system32>net stop MongoDB
El servicio de MongoDB Server (MongoDB) está deteniéndose.
El servicio de MongoDB Server (MongoDB) se detuvo correctamente.
```

El **puerto de escucha** por defecto es el **27017**, y podemos comprobar que el servidor está escuchando desde un terminal CMD con el comando `netstat` .

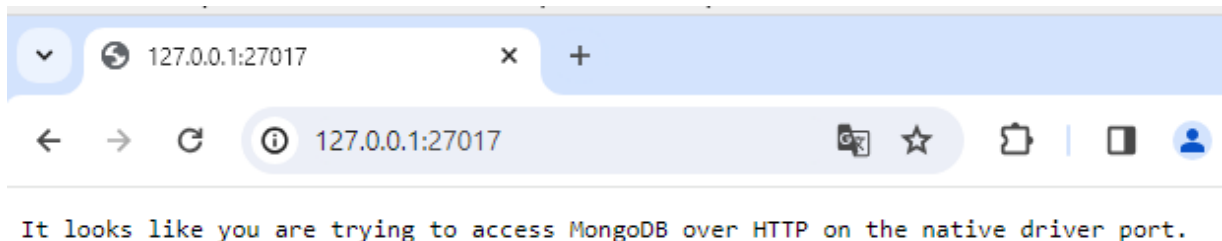
Para sistemas Windows utilizaremos:

```
netstat -a -p TCP
```

Para sistemas Linux utilizaremos:

```
netstat -at
```

Podemos comprobar el funcionamiento del servidor MongoDB abriendo un navegador y accediendo a la URL <http://localhost:27017>. Si todo funciona correctamente, veremos un mensaje similar al siguiente:



⚡ Usuarios de Windows 10

MongoDB declara que Windows 10 es un sistema operativo soportado para las ediciones Community y Enterprise. Pero lo importante es que en MongoDB 8.0 se introdujo una actualización profunda de TCMalloc (el gestor de memoria), y según la documentación oficial:

Windows uses the legacy TCMalloc version and does not support the upgraded TCMalloc introduced in MongoDB 8.0.

Muchos usuarios han informado de fallos relacionados con esto, específicamente crashes en tcmalloc con excepción 0xC000001D. Por lo tanto, hay que instalar una versión de MongoDB anterior a la 8.0 para evitar estos problemas.

¿ Cómo acceder desde otro equipo ?

Por defecto, MongoDB solo escucha en la dirección local **127.0.0.1**, como se puede comprobar mediante el comando `netstat`. Si necesitas acceder al servidor desde otro equipo, deberás habilitar las conexiones remotas.

En **Linux**, el archivo de configuración se encuentra en: **/etc/mongod.conf**

En **Windows**, se ubica en: **C:\Program Files\MongoDB\Server\X.X\bin\mongod.cfg** donde X.X es la versión instalada.

Dentro de este archivo debes localizar la línea `bindIp: 127.0.0.1` y sustituirla por `bindIp: 0.0.0.0`. Este cambio indica al servicio que acepte conexiones desde cualquier dirección IP.



- [Documentación oficial MongoDB](#)

Entorno de trabajo en MongoDB

Una vez instalado MongoDB, podemos interactuar con el servidor de varias formas. La más directa es mediante la consola de comandos **MongoDB Shell (mongosh)**, que se instala automáticamente junto con el servidor MongoDB. También existen herramientas gráficas como **MongoDB Compass** o **MongoAdmin** que facilitan la gestión visual de las bases de datos. En nuestro caso hemos instalado **MongoDB Compass** junto con el servidor, por lo que podemos utilizarlo para conectarnos al servidor y gestionar las bases de datos de forma visual. Tanto MongoDB Compass como MongoDB Shell se pueden utilizar para conectarse a servidores MongoDB locales o remotos.

En las **instalaciones como localhost se deshabilita la autenticación** por defecto, por lo que no es necesario crear usuarios para conectarse al servidor. En instalaciones en producción, es recomendable habilitar la autenticación y crear usuarios con los permisos adecuados para cada tarea.

MongoDB Compass

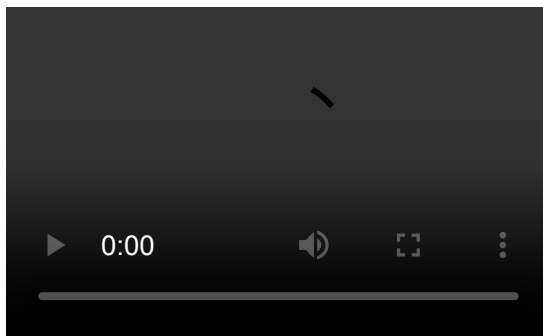
MongoDB Compass es una herramienta gráfica de escritorio que permite a los usuarios interactuar con sus bases de datos MongoDB de manera visual e intuitiva. Proporciona una interfaz amigable para explorar, consultar y administrar las colecciones y documentos almacenados en MongoDB, sin necesidad de escribir comandos en la línea de comandos.

MongoDB Compass se puede descargar gratuitamente desde la página oficial de MongoDB (<https://www.mongodb.com/try/download/compass>), y está disponible para Windows, macOS y Linux.

Con MongoDB Compass, los usuarios pueden conectarse a sus despliegues de MongoDB, ya sea localmente o en la nube, y realizar operaciones como crear bases de datos, colecciones y documentos, ejecutar consultas con filtros avanzados, visualizar índices y analizar el rendimiento de las consultas. Además, ofrece funcionalidades para importar y exportar datos, así como para gestionar usuarios y permisos.

Utilizar por primera vez MongoDB Compass es sencillo. Al abrir la aplicación, se presenta una pantalla de conexión donde se pueden ingresar los detalles del servidor MongoDB al que se desea conectar, como la dirección IP, el puerto, las credenciales de autenticación y otras opciones de configuración. Una vez establecida la conexión, se accede a una interfaz gráfica que muestra las bases de datos disponibles, permitiendo navegar por ellas y realizar diversas operaciones de gestión y consulta de manera visual.

El siguiente video muestra los pasos iniciales para conectarse a un servidor MongoDB local utilizando MongoDB Compass.



Como se muestra en el video, podemos abrir un terminal de Mongo Shell desde MongoDB Compass.

MongoDB Shell (mongosh)

MongoDB Shell, conocido como **mongosh**, es un entorno interactivo REPL basado en JavaScript y Node.js que permite conectarse y trabajar directamente con despliegues de MongoDB, ya sea a nivel local, remoto o en la nube mediante Atlas. Permite ejecutar consultas, administrar bases de datos, manipular datos mediante operaciones CRUD y automatizar tareas mediante scripts. Su diseño moderno mejora significativamente la experiencia del desarrollador gracias a funcionalidades como autocompletado, resaltado de sintaxis y mensajes de error más detallados.

No es necesario instalar mongosh en el mismo equipo donde se encuentra el servidor MongoDB, ya que puede instalarse de forma independiente en cualquier máquina que necesite conectarse al servidor. Esto facilita la administración remota y el desarrollo distribuido. Si instalamos mongosh en un equipo diferente, debemos asegurarnos de que el servidor MongoDB esté configurado para aceptar conexiones remotas, ajustando la directiva `bindIp` con el valor `0.0.0.0` en el archivo de configuración `mongod.cfg`. Si tenemos algún cortafuegos o firewall, también debemos asegurarnos de que el puerto 27017 esté abierto para permitir la comunicación entre mongosh y el servidor MongoDB.

Para descargar e instalar mongosh de forma independiente, podemos visitar la página oficial de descargas en <https://www.mongodb.com/try/download/shell> y seguir las instrucciones específicas para nuestro sistema operativo.

Una vez instalado, podemos conectarnos al servidor MongoDB utilizando la línea de comandos.

```
mongosh mongodb://<IP_DEL_SERVIDOR>:27017
```

Donde `<IP_DEL_SERVIDOR>` es la dirección IP del servidor MongoDB al que queremos conectarnos. Si el servidor está configurado correctamente, deberíamos ver un mensaje de bienvenida indicando que la conexión se ha establecido con éxito.

Instalar mongosh en Linux

En este ejemplo, descargaremos e instalaremos mongosh en un equipo con Debian 11 Linux y nos conectaremos a un servidor MongoDB de nuestra red local (192.168.2.5) que corre en una máquina con Windows 10. Vamos a suponer que el servidor MongoDB está configurado para aceptar conexiones de otros equipos de la red local y que previamente se ha creado una regla en el cortafuegos de Windows para permitir las conexiones entrantes al puerto 27017. Con estos supuestos, los pasos para descargar, instalar y conectarnos al servidor remoto son los siguientes:

- **Obtener URL de descarga.** Accedemos a la página web de descargas de mongosh (<https://www.mongodb.com/try/download/shell>) y vemos cual es la url para descargar el paquete DEB para Linux x64.
- **Descargar e instalar.** Descargamos el paquete y lo instalamos con el gestor de paquetes proporcionado por la distribución.

Para el caso de equipos sin entorno gráfico, podemos realizar la descarga e instalación desde la línea de comandos:

```
wget https://downloads.mongodb.com/compass/mongodb-mongosh_2.6.0_amd64.deb
dpkg -i mongodb-mongosh_2.6.0_amd64.deb
```

- **Conectarse al servidor remoto.** Una vez instalado, podemos conectarnos al servidor remoto desde una terminal el siguiente comando:

```
mongosh mongodb://192.168.2.5:27017
```

Si nos aparece algo similar a la siguiente imagen, significa que la conexión se ha realizado correctamente.

```
root@PCL-1:~# mongosh mongodb://192.168.2.5:27017
Current Mongosh Log ID: 698397831be432b0de8ce5af
Connecting to:      mongodb://192.168.2.5:27017/?directConnection=true&appName=mongosh+2.6.0
Using MongoDB:      7.0.29
Using Mongosh:       2.6.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-02-04T19:37:14.150+01:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> _
```

Instalar mongosh en Windows

En este supuesto, instalaremos mongosh en un equipo con Windows 10 y nos conectaremos al servidor MongoDB del ejemplo anterior (192.168.2.5).

El proceso es muy sencillo, solo tenemos que descargarnos el instalador MSI desde la página oficial de descargas (<https://www.mongodb.com/try/download/shell>) e instalarlo.

A continuación, abrimos una terminal CMD y nos conectamos al servidor remoto con el siguiente comando:

```
mongosh mongodb://192.168.2.5:27017
```

Si todo ha ido bien, nos aparecerá una pantalla similar a la del ejemplo anterior, indicando que la conexión se ha realizado correctamente.

```
C:\Users\usuario>mongosh mongodb://192.168.2.5:27017
Current Mongosh Log ID: 6983a69defb45c9b3c628c9f
Connecting to:  mongodb://192.168.2.5:27017/?directConnection=true&appName=mongosh+2.6.0
Using MongoDB:  7.0.29
Using Mongosh:  2.6.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-02-04T21:05:09.846+01:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> _
```

Bases de datos y colecciones en MongoDB

Antes de profundizar en la estructura de los documentos y los tipos de datos, es importante entender cómo organiza MongoDB la información.

En MongoDB, una **base de datos (database)** es un contenedor lógico que agrupa varias colecciones. Cada base de datos mantiene su propio espacio de nombres y su propio conjunto de archivos de almacenamiento. Un solo servidor MongoDB puede gestionar múltiples bases de datos simultáneamente.

Dentro de una base de datos encontramos las **colecciones (collections)**, que son agrupaciones de documentos. A diferencia de las tablas en los sistemas relacionales, las colecciones no requieren un esquema fijo: los documentos almacenados pueden tener campos distintos y estructuras variadas. Esto proporciona una gran flexibilidad en el diseño de la información, especialmente útil en aplicaciones que evolucionan con rapidez.



Base de datos y colecciones

Imaginemos una base de datos llamada `empresa`.

Dentro de ella podría haber varias colecciones:

- `empleados`
- `departamentos`
- `proyectos`

Cada colección contiene documentos independientes, sin necesidad de un esquema preestablecido.

Estructura de un documento MongoDB

En MongoDB, la unidad fundamental de almacenamiento es el **documento**, un conjunto de **pares clave–valor** que se asemeja conceptualmente a un objeto JSON. Sin embargo, la representación interna se realiza mediante **BSON (Binary JSON)**, un formato binario optimizado que permite manejar datos de forma más eficiente y soportar tipos adicionales.

Diferencias entre JSON y BSON

Característica	JSON	BSON
Formato	Texto	Binario
Tipos de datos	Limitados (string, number, boolean, null, array, object)	Ampliados (incluye tipos binarios, fechas, ObjectId, etc.)
Claves y valores	Requiere comillas siempre	Solo se requieren comillas para valores string
Eficiencia	Menos eficiente para lectura/escritura	Más eficiente debido a su formato binario

Un documento MongoDB tiene las siguientes características principales:

- Es **semiestructurado**, lo que significa que cada documento puede poseer claves y tipos distintos.
- Es **jerárquico**, permitiendo anidar documentos dentro de otros documentos.
- Es **flexible**, ya que su estructura no está definida por un esquema rígido.
- Es **autocontenido**, pues cada documento incluye su propia estructura y metadatos implícitos.
- Permite **arrays** con elementos homogéneos o heterogéneos.
- Se identifica normalmente mediante un campo `_id`, de tipo `objectId` por defecto.



A continuación se muestra un ejemplo de documento MongoDB que representa la información de una persona:

```
{
  "_id": ObjectId("65a2c3109d1cce001fa9d8b2"),
  "nombre": "María",
  "edad": 29,
  "direccion": {
    "calle": "Gran Vía",
    "numero": 10,
    "ciudad": "Madrid"
  },
  "aficiones": ["cine", "lectura", "senderismo"],
  "activo": true
}
```

Como se puede observar, el documento contiene varias claves con diferentes tipos de datos: un `ObjectId` para el campo `_id`, un `string` para `nombre`, un `int32` para `edad`, un subdocumento para `direccion`, un array de strings para `aficiones` y un booleano para `activo`.

Como no es necesario que las claves vayan entre comillas, el documento también podría escribirse de la siguiente forma:

```
{
  _id: ObjectId("65a2c3109d1cce001fa9d8b2"),
  nombre: "María",
  edad: 29,
  direccion: {
    calle: "Gran Vía",
    numero: 10,
    ciudad: "Madrid"
  },
  aficiones: ["cine", "lectura", "senderismo"],
  activo: true
}
```

Tipos de datos

BSON extiende JSON añadiendo una tipología más rica y una codificación binaria eficiente. A continuación se presenta un compendio sistemático de los **tipos de datos** más relevantes, su semántica, casos de uso y observaciones relacionadas con su representación.

Tipos primitivos

String (UTF-8)

Representa texto en codificación UTF-8. Es el tipo más habitual para campos descriptivos, nombres de usuario o contenido textual.

String

El siguiente ejemplo muestra un documento con un campo de tipo String.

```
{ "nombre": "Lucía" }
```

O alternativamente, sin comillas en la clave:

```
{ nombre: "Lucía" }
```

Number

BSON distingue entre cuatro tipos numéricos optimizados para distintos rangos y precisiones:

- **int32**: entero con signo de 32 bits.
- **int64 / NumberLong**: entero con signo de 64 bits.
- **double**: número de coma flotante (IEEE 754).
- **decimal128**: número decimal de alta precisión, especialmente útil en cálculos financieros.

Tipos numéricos

El siguiente ejemplo muestra un documento con varios campos numéricos.

```
{  
  "edad": 30,  
  "poblacion": NumberLong(9000000000),  
  "temperatura": 22.5,  
  "saldo": NumberDecimal("1200.75")  
}
```

El campo `edad` es un entero de 32 bits, `poblacion` es un entero de 64 bits, `temperatura` es un número de coma flotante y `saldo` es un decimal de alta precisión.

Boolean

Valores lógicos `true` y `false` .

Boolean

El siguiente ejemplo muestra un documento con un campo booleano.

```
{ "activo": true }
```

El valor Null

El valor null no es un tipo de dato realmente, se utiliza para representar un valor intencionalmente vacío y se utiliza para indicar la ausencia de un valor. Es aplicable a cualquier tipo de dato.

Valor nulo

El siguiente ejemplo muestra un documento con un campo que tiene un valor nulo.

```
{ telefono: null }
```

Tipos complejos

Document (Documento anidado)

Estructura jerárquica compuesta por pares clave–valor. Permite modelar subentidades (relaciones 1:1 y 1:N dentro del propio documento), evitando la necesidad de *joins*.

Documento anidado

El siguiente ejemplo muestra un documento con un subdocumento `direccion` y otro `empresa` .

```
{
  "nombre": "Sergio",
  "direccion": {
    "calle": "Alameda",
    "numero": 22,
    "ciudad": "Valencia",
    "cp": "46001"
  },
  "empresa": {
    "nombre": "TechNova",
    "cif": "B12345678"
  }
}
```

Array

Colección ordenada cuyos elementos pueden ser homogéneos (todos del mismo tipo) o heterogéneos (mezcla de diferentes tipos de datos). Muy útil para listas de etiquetas, historiales y colecciones embebidas.

Se utilizan los corchetes `[]` para definir arrays y los elementos se separan por comas. El acceso a los elementos se realiza mediante índices basados en cero.

Array

Este ejemplo muestra un array heterogéneo con strings y subdocumentos.

```
{
  "usuario": "ana.gonzalez",
  "aficiones": ["fotografia", "yoga", {"tipo": "lectura", "frecuencia": "semanal"}]
}
```

Tipos específicos (especializados)

ObjectId

Identificador único de 12 bytes que se asigna por defecto al campo `_id`. El campo `_id` es un `ObjectId` que se genera automáticamente si no se proporciona uno al insertar el documento (equivalente a los campos autoincrementados en bases de datos relacionales). Este tipo de dato es especialmente útil para garantizar la unicidad de los documentos dentro de una colección y para optimizar las consultas basadas en este identificador.

ObjectId

Documento con `_id` autogenerado y un campo adicional.

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "titulo": "Introducción a MongoDB"
}
```

El `_id` es inmutable, lo que significa que una vez asignado a un documento, no puede ser modificado. Esto garantiza la integridad de los datos y permite que el sistema de indexación de MongoDB funcione de manera eficiente.

Aunque no es aconsejable, es posible asignar un valor personalizado al campo `_id` al insertar un documento. Sin embargo, si se opta por esta práctica, se debe asegurar que el valor asignado sea único dentro de la colección para evitar conflictos ya que si se intenta insertar un documento con un `_id` que ya existe, MongoDB generará un error de duplicidad.

Date

Representa fecha y hora en milisegundos desde la época UNIX (1 de enero de 1970). Se utiliza para almacenar marcas de tiempo en aplicaciones. Las fechas se almacenan en UTC y se recomienda siempre trabajar con esta zona horaria para evitar confusiones.

Si estamos en España y es invierno (UTC+1), al almacenar una fecha en MongoDB tenemos que restar una hora para que se almacene correctamente en UTC. Sin embargo, si es verano (UTC+2), debemos restar dos horas.

Si lo que queremos es recuperar la fecha en la zona horaria local, debemos sumar una hora en invierno y dos horas en verano al valor almacenado en MongoDB.

Date

Documento con una marca de tiempo de creación. Esto es lo que tendríamos que haber asignado a `fecha_creacion` si estamos en España, es invierno y son las 15:30 horas en horario local.

```
{
  titulo: "Entrada de blog",
  fecha_creacion: ISODate("2024-11-05T14:30:00Z")
}
```

Alternativamente, podríamos haber proporcionado el offset (`±hh:mm`) de la zona horaria local, lo que indicaría que la fecha se ha proporcionado en la zona horaria local. En ese caso, MongoDB interpretaría la fecha como si estuviera en la zona horaria local y la convertiría a UTC automáticamente al almacenarla.

```
{
  titulo: "Entrada de blog",
  fecha_creacion: ISODate("2024-11-05T15:30:00-01:00")
}
```

Además de los tipos mencionados, BSON incluye otros tipos especializados como **Timestamp**, **Binary Data**, **Regular Expression**, **Code** y los valores centinela **MinKey** y **MaxKey**.

Operadores en MongoDB

MongoDB proporciona una amplia gama de operadores que permiten realizar consultas, actualizaciones y manipulaciones de datos de manera eficiente. Estos operadores se clasifican en varias categorías según su función:

- **Operadores de consulta:** utilizados para filtrar documentos según criterios específicos. Tenemos operadores de comparación, lógicos, de evaluación de elementos, de expresión regular, entre otros.
 - **Operadores relacionales:** permiten comparar valores en consultas. Algunos de los operadores relacionales más comunes son:

Operador	Descripción
<code>\$eq</code>	Igual a
<code>\$ne</code>	No igual a
<code>\$gt</code>	Mayor que
<code>\$gte</code>	Mayor o igual que
<code>\$lt</code>	Menor que
<code>\$lte</code>	Menor o igual que
<code>\$in</code>	En un conjunto de valores
<code>\$nin</code>	No en un conjunto de valores

- **Operadores lógicos:** permiten combinar múltiples condiciones en una consulta. Algunos de los operadores lógicos más comunes son:

Operador	Descripción
<code>\$and</code>	Y lógico (todas las condiciones deben ser verdaderas)
<code>\$or</code>	O lógico (al menos una condición debe ser verdadera)
<code>\$not</code>	Negación lógica (invierte el resultado de una condición)
<code>\$nor</code>	NOR lógico (ninguna de las condiciones debe ser verdadera)

- **Operadores de evaluación de elementos:** permiten evaluar la existencia o el tipo de campos en los documentos. Algunos de los operadores de evaluación más comunes son:

Operador	Descripción
<code>\$exists</code>	Verifica la existencia de un campo
<code>\$type</code>	Verifica el tipo de un campo

- **Operadores de expresión regular:** permiten realizar búsquedas basadas en patrones de texto. Algunos de los operadores de expresión regular más comunes son:

Operador	Descripción
<code>\$regex</code>	Coincide con una expresión regular
<code>\$options</code>	Especifica opciones para la expresión regular (como <code>i</code> para insensible a mayúsculas)

- **Operadores de actualización:** utilizados para modificar documentos existentes. Algunos de los operadores de actualización más comunes son:

Operador	Descripción
<code>\$set</code>	Establece el valor de un campo
<code>\$unset</code>	Elimina un campo
<code>\$inc</code>	Incrementa el valor de un campo numérico
<code>\$push</code>	Agrega un elemento a un array
<code>\$pull</code>	Elimina elementos de un array que coincidan con una condición

- **Operadores de agregación:** utilizados en el marco de agregación para transformar y analizar datos. Algunos de los operadores de agregación más comunes son:

Operador	Descripción
<code>\$match</code>	Filtra documentos según criterios de consulta
<code>\$group</code>	Agrupar documentos por un campo específico y realiza operaciones de agregación
<code>\$sort</code>	Ordena los documentos según uno o más campos
<code>\$project</code>	Modifica la forma de los documentos en la salida, incluyendo o excluyendo campos
<code>\$limit</code>	Limita el número de documentos en la salida
<code>\$unwind</code>	Descompone un array en documentos individuales
<code>\$lookup</code>	Realiza una operación de join con otra colección

- **Operadores aritméticos:** utilizados para realizar operaciones matemáticas en consultas de agregación. Algunos de los operadores aritméticos más comunes son:

Operador	Descripción
<code>\$add</code>	Suma dos números o campos
<code>\$subtract</code>	Resta dos números o campos
<code>\$multiply</code>	Multiplica dos números o campos
<code>\$divide</code>	Divide dos números o campos
<code>\$mod</code>	Calcula el módulo de dos números o campos
<code>\$abs</code>	Calcula el valor absoluto de un número o campo
<code>\$ceil</code>	Redondea un número hacia arriba
<code>\$floor</code>	Redondea un número hacia abajo
<code>\$round</code>	Redondea un número a un número específico de decimales
<code>\$trunc</code>	Trunca un número a un número específico de decimales
<code>\$rand</code>	Genera un número aleatorio entre 0 y 1

- **Operadores de String:** utilizados para realizar operaciones de manipulación de cadenas en consultas de agregación. Algunos de los operadores de string más comunes son:

Operador	Descripción
<code>\$concat</code>	Concatenar dos o más cadenas
<code>\$substr</code>	Extraer una subcadena de una cadena
<code>\$toUpper</code>	Convertir una cadena a mayúsculas
<code>\$toLower</code>	Convertir una cadena a minúsculas
<code>\$trim</code>	Eliminar espacios en blanco de ambos extremos de una cadena
<code>\$size</code>	Obtener la longitud de una cadena
<code>\$split</code>	Dividir una cadena en un array de subcadenas
<code>\$strcasecmp</code>	Comparar dos cadenas sin distinguir mayúsculas de minúsculas

- **Operadores de fecha:** utilizados para realizar operaciones de manipulación de fechas en consultas de agregación. Algunos de los operadores de fecha más comunes son:

Operador	Descripción
<code>\$dateToString</code>	Convertir una fecha a una cadena con un formato específico
<code>\$dateFromString</code>	Convertir una cadena a una fecha según un formato específico
<code>\$dateAdd</code>	Agregar un número específico de unidades de tiempo a una fecha
<code>\$dateSubtract</code>	Restar un número específico de unidades de tiempo a una fecha
<code>\$dateDiff</code>	Calcular la diferencia entre dos fechas en unidades específicas
<code>\$year</code>	Extraer el año de una fecha
<code>\$month</code>	Extraer el mes de una fecha
<code>\$dayOfMonth</code>	Extraer el día del mes de una fecha
<code>\$dayOfWeek</code>	Extraer el día de la semana de una fecha

Esta es solo una selección de los operadores más comunes en MongoDB. La plataforma ofrece una amplia variedad de operadores adicionales para cubrir una gran cantidad de casos de uso y necesidades específicas.

Hay que tener en cuenta que algunos de estos operadores solo están disponibles en el marco de agregación, mientras que otros pueden ser utilizados con cualquiera de los métodos de consulta o actualización. Es importante consultar la documentación oficial de MongoDB para conocer la sintaxis exacta y las opciones disponibles para cada operador.

Gestión de bases de datos y colecciones

Trabajar con MongoDB implica interactuar con el sistema a través de mongosh, la consola oficial del SGBD. Aunque MongoDB es una base de datos orientada a documentos, sus comandos mantienen cierta similitud conceptual con los del SQL tradicional: seleccionar bases de datos, consultar información, insertar documentos... pero con una sintaxis y filosofía propia del modelo NoSQL. A continuación se presentan los comandos más habituales, acompañados de explicaciones y ejemplos que ilustran su función dentro del flujo de desarrollo.

Comandos relacionados con bases de datos

Tenemos varios comandos relacionados con la gestión de bases de datos, que nos permiten crear, seleccionar y consultar las bases de datos disponibles en el servidor.

Crear o seleccionar una base de datos

En MongoDB, el comando `use` permite cambiar el contexto a una base de datos concreta. Si la base de datos no existe, se creará automáticamente cuando añadas la primera colección. Su sintaxis es la siguiente:

```
use <nombreBD>
```



Seleccionar base de datos

Para seleccionar la base de datos llamada `empresa`, se ejecutaría:

```
use empresa
```

Consultar la base de datos activa

El comando `db` muestra la base de datos actualmente seleccionada en el contexto de la sesión. Su sintaxis es simple:

```
db
```

Mostrar las bases de datos disponibles

El comando `show dbs` lista todas las bases de datos presentes en el servidor MongoDB. Solo aquellas que contienen datos serán mostradas. Su sintaxis es la siguiente:

```
show dbs
```

Comandos relacionados con colecciones

Las colecciones equivalen a las tablas del modelo relacional y agrupan documentos.

Mostrar las colecciones de la base de datos actual

El comando `show collections` lista todas las colecciones dentro de la base de datos actualmente seleccionada. Su sintaxis es la siguiente:

```
show collections
```

Crear colecciones

Aunque MongoDB crea automáticamente una colección al insertar el primer documento, también es posible crear una colección explícitamente utilizando el comando `createCollection`. Su sintaxis es la siguiente:

```
db.createCollection("<nombreColeccion>")
```



Crear colección

Para crear una colección llamada `empleados`, se ejecutaría:

```
db.createCollection("empleados")
```

Eliminar una colección

El comando `drop` elimina una colección completa junto con todos sus documentos. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.drop()
```



Eliminar colección

Para eliminar la colección llamada `empleados`, se ejecutaría:

```
db.empleados.drop()
```

Las colecciones almacenan documentos, que sería el equivalente a las filas en una tabla relacional. En las siguientes secciones veremos cómo realizar consultas y actualizaciones sobre los documentos de una colección.

Consulta de documentos en colecciones

Tenemos dos formas diferentes de realizar consultas sobre documentos:

- **Consulta directa:** permite recuperar documentos de una colección utilizando el comando `find` o `findOne`.
- **Aggregation Framework:** proporciona un conjunto de operadores y etapas para realizar consultas más complejas, transformaciones y cálculos sobre los datos.

Consultas directas

Consultas elementales

El comando `find` nos permite recuperar documentos de una colección. Este comando puede aceptar criterios de búsqueda para filtrar los resultados. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.find({<criterio>})
```

donde `<criterio>` es un objeto en formato JSON que especifica las condiciones que deben cumplir los documentos para ser incluidos en el resultado. Si no se proporciona ningún criterio, el comando `find` devolverá todos los documentos de la colección.



Comando find

Para obtener todos los documentos de la colección `empleados`, se ejecutaría:

```
db.empleados.find()
```

Para filtrar empleados del departamento de *Ventas*, se ejecutaría:

```
db.empleados.find(  
  { "departamento": "Ventas" }  
)
```

El comando `findOne` recupera un único documento que coincida con el criterio especificado. Este comando devuelve el primer documento que coincide con el criterio proporcionado. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.findOne({<criterio>})
```



Comando findOne

Para obtener un solo documento de la colección `empleados`, se ejecutaría:

```
db.empleados.findOne(  
  { "nombre": "Ana" }  
)
```

Con los comandos `find` y `findOne`, es posible utilizar el método `.pretty()` para formatear la salida de los documentos de manera más legible.

```
db.<nombreColeccion>.find().pretty()
```



Formatear salida de documentos

Para formatear la salida de los documentos de la colección `empleados`, se ejecutaría:

```
db.empleados.find().pretty()
```

Selección de campos en consultas (proyección)

Para especificar qué campos se desean incluir o excluir en los resultados de una consulta, se puede utilizar el segundo argumento del método `find()`, conocido como **proyección**. Este argumento es un objeto que indica con un valor de 1 los campos que se desean incluir y con un valor de 0 los campos que se desean excluir. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.find({<criterio>}, {<campo1>: <incluir|excluir>, <campo2>: <incluir|excluir>, ...})
```

donde `incluir` se representa con el valor 1 y `excluir` con el valor 0.

Selección de campos

Para obtener solo los nombres y edades de los empleados, se ejecutaría:

```
db.empleados.find(
  {},
  { "nombre": 1, "edad": 1, "_id": 0 }
)
```

En este ejemplo, se incluyen los campos `nombre` y `edad`, y se excluye el campo `_id` que se incluye por defecto.

Condiciones con operadores relacionales

Vamos a ver como se pueden utilizar operadores de comparación y lógicos para construir consultas más complejas. MongoDB soporta una amplia variedad de operadores que permiten filtrar documentos en función de condiciones específicas.

Condiciones simples

Para filtrar empleados mayores de 30 años, se ejecutaría:

```
db.empleados.find(
  { "edad": { $gt: 30 } }
)
```

Para filtrar empleados que no pertenecen al departamento de *Ventas*, se ejecutaría:

```
db.empleados.find(
  { "departamento": { $ne: "Ventas" } }
)
```

Condiciones simples

Para filtrar empleados que tienen una edad entre 25 y 35 años, se ejecutaría:

```
db.empleados.find(
  { "edad": { $gte: 25, $lte: 35 } }
)
```

Para filtrar empleados que pertenecen a los departamentos de *Ventas* o *Marketing*, se ejecutaría:

```
db.empleados.find(
  { "departamento": { $in: ["Ventas", "Marketing"] } }
)
```

Se pueden combinar múltiples condiciones utilizando operadores lógicos como `$and`, `$or` y `$not`. En este caso, el criterio de búsqueda se define como un objeto que contiene un operador lógico con una lista (array) de condiciones.



Condiciones compuestas

Para filtrar empleados que son mayores de 30 años y pertenecen al departamento de *Ventas*, se ejecutaría:

```
db.empleados.find(
  {
    $and: [
      { "edad": { $gt: 30 } },
      { "departamento": "Ventas" }
    ]
  }
)
```

Para filtrar empleados que son mayores de 30 años o pertenecen al departamento de *Ventas*, se ejecutaría:

```
db.empleados.find(
  {
    $or: [
      { "edad": { $gt: 30 } },
      { "departamento": "Ventas" }
    ]
  }
)
```


Expresiones regulares

Con MongoDB también es posible utilizar expresiones regulares para realizar búsquedas de patrones en campos de tipo string. Esto es especialmente útil para filtrar documentos basados en coincidencias parciales o patrones específicos. La sintaxis para utilizar expresiones regulares en una consulta es la siguiente:

```
db.<nombreColeccion>.find({<campo>: { $regex: <expresión_regular> }})
```

De manera alternativa, también se puede utilizar la sintaxis de expresiones regulares de JavaScript:

```
db.<nombreColeccion>.find({<campo>: /<expresión_regular>/})
```

El símbolo `^` se utiliza para indicar que la coincidencia debe ocurrir al inicio de la cadena, mientras que el símbolo `$` indica que la coincidencia debe ocurrir al final de la cadena.

Expresiones regulares

Para filtrar empleados cuyo nombre comienza con la letra "A", se ejecutaría:

```
db.empleados.find(
  { "nombre": { $regex: "^A" } }
)
```

Alternativamente, también se puede utilizar la sintaxis de expresiones regulares de JavaScript:

```
db.empleados.find(
  { "nombre": /^A/ }
)
```

Expresiones regulares

Para filtrar empleados cuyo nombre termina con la letra "a", se ejecutaría:

```
db.empleados.find(
  { "nombre": { $regex: "a$" } }
)
```

Consultas multietapa y encadenamiento de métodos

Las consultas en MongoDB se pueden ejecutar en varias etapas de manera encadenada, lo que permite aplicar múltiples operaciones sobre los resultados de una consulta. Por ejemplo, después de filtrar los documentos con `find()`, es posible ordenar los resultados, limitar el número de documentos devueltos o seleccionar campos específicos.

A continuación veremos cómo se pueden encadenar estas operaciones para realizar consultas más complejas.

Ordenamiento de resultados

Para ordenar los resultados de una consulta, se puede utilizar el método `sort()`. Este método acepta un objeto que especifica los campos por los que se desea ordenar y el orden (1 para ascendente y -1 para descendente). Su sintaxis es la siguiente:

```
db.<nombreColeccion>.find(<{<criterio>}>).sort(<{<campo1>: <orden1>, <campo2>: <orden2>, ...}>)
```



Ordenar resultados

Para ordenar los empleados por edad de forma descendente

```
db.empleados.find().sort({ "edad": -1 })
```

Para ordenar los empleados por edad de forma descendente y luego por nombre de forma ascendente, se ejecutaría:

```
db.empleados.find().sort({ "edad": -1, "nombre": 1 })
```

Limitar el número de resultados

Para limitar el número de documentos devueltos por una consulta, se puede utilizar el método `limit()`. Este método acepta un número entero que especifica la cantidad máxima de documentos a retornar. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.find(<{<criterio>}>).limit(<número>)
```



Limitar resultados

Para obtener solo los 5 empleados más jóvenes, se ejecutaría:

```
db.empleados.find().sort({ "edad": 1 }).limit(5)
```

También es posible combinar `limit()` con `skip()` para paginar los resultados de una consulta. El método `skip()` permite omitir un número específico de documentos antes de comenzar a retornar los resultados. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.find({<criterio>}).skip(<número>).limit(<número>)
```



Paginación de resultados

Para obtener los empleados del segundo grupo de 5 empleados más jóvenes, se ejecutaría:

```
db.empleados.find().sort({ "edad": 1 }).skip(5).limit(5)
```

Consultas sobre arrays

MongoDB ofrece operadores específicos para consultar documentos que contienen campos de tipo array. Estos operadores permiten filtrar documentos basados en la presencia de elementos específicos dentro de un array, la coincidencia de patrones o la cantidad de elementos.

Algunos de los operadores más comunes para consultas sobre arrays son:

Operador	Descripción
\$elemMatch	Filtra documentos que contienen un array con al menos un elemento que cumple con las condiciones especificadas.
\$size	Filtra documentos que contienen un array con un número específico de elementos.
\$all	Filtra documentos que contienen un array que incluye todos los elementos especificados.
\$in	Filtra documentos que contienen un array con al menos un elemento que coincide con los valores especificados.
\$nin	Filtra documentos que contienen un array sin ningún elemento que coincida con los valores especificados.



Consultas sobre arrays

Para filtrar empleados que tienen "yoga" como una de sus aficiones, se ejecutaría:

```
db.empleados.find(
  { "aficiones": { $elemMatch: { $eq: "yoga" } } }
)
```

Para filtrar empleados que tienen exactamente 3 aficiones, se ejecutaría:

```
db.empleados.find(
  { "aficiones": { $size: 3 } }
)
```

Para filtrar empleados que tienen tanto "cine" como "yoga" entre sus aficiones, se ejecutaría:

```
db.empleados.find(
  { "aficiones": { $all: ["cine", "yoga"] } }
)
```

Uso del metodo forEach

El método `forEach()` se puede utilizar para iterar sobre los documentos devueltos por una consulta y realizar operaciones específicas en cada documento. Este método acepta una función de callback que se ejecuta para cada documento en el cursor de resultados. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.find({<criterio>}).forEach(function(documento) {
  // Operaciones a realizar con cada documento
})
```

Es habitual pasar una función anónima como callback, aunque también se puede definir una función por separado y luego pasarla como argumento a `forEach()`.

```
db.<nombreColeccion>.find({<criterio>}).forEach(documento => {
  // Operaciones a realizar con cada documento
})
```

Un uso común del método `forEach()` es imprimir unos campos específicos de cada documento en la consola. Para ello, se puede utilizar la función `print()` dentro del callback. La salida sería similar a la que muestran las consultas en las bases de datos relacionales, pero con la flexibilidad de poder formatear la salida a nuestro gusto.



Uso del método forEach

Queremos saber qué equipos de nuestra base de datos de la NBA pertenecen a la división *SouthEast* y mostrar su `nombre` y `ciudad` .

Si no utilizamos `forEach()` , la consulta sería la siguiente:

```
db.equipos.find({Division: "SouthEast"}, {_id:0, Nombre:1, Ciudad:1})
```

y nos mostraría la salida como una lista de documentos JSON:

```
[
  { Nombre: 'Bobcats', Ciudad: 'Charlotte' },
  { Nombre: 'Hawks', Ciudad: 'Atlanta' },
  { Nombre: 'Heat', Ciudad: 'Miami' },
  { Nombre: 'Magic', Ciudad: 'Orlando' },
  { Nombre: 'Wizards', Ciudad: 'Washington' }
]
```

Si utilizamos `forEach()` , podemos mostrar solo el nombre y la ciudad de cada equipo, separados por una tabulación, lo que nos daría una salida más limpia y fácil de leer:

```
db.equipos.find({Division: "SouthEast"}, {_id:0, Nombre:1, Ciudad:1}).forEach(doc => print(doc.Nombre +
```

Esto nos mostraría lo siguiente:

```
Bobcats  Charlotte
Hawks    Atlanta
Heat     Miami
Magic    Orlando
Wizards  Washington
```

Consultas de agregación (Aggregation Framework)

El **Aggregation Framework** de MongoDB permite realizar consultas avanzadas y transformar datos mediante un conjunto de etapas encadenadas, conocidas como *pipeline*. En el procesamiento *pipeline* el resultado de cada etapa se pasa a la siguiente que lo utiliza como entrada. Cada etapa aplica una operación específica, como filtrado (`$match`), agrupamiento (`$group`), ordenamiento (`$sort`) o proyección (`$project`), facilitando el análisis y procesamiento flexible de la información.

Para utilizarlo, se emplea el método `aggregate()` , que recibe un array de etapas. El resultado de cada etapa se pasa a la siguiente, permitiendo construir consultas complejas:

```
db.<nombreColeccion>.aggregate([
  { <etapa1>: { <operadores> } },
  { <etapa2>: { <operadores> } },
  // ...
])
```

Este enfoque supera las capacidades de las consultas directas, ya que permite transformar, agrupar y analizar los datos de forma personalizada y eficiente.

Todo lo que se puede hacer con `find()` también se puede realizar con `aggregate()`.

Selección de campos con `$project` :

```
db.<nombreColeccion>.aggregate([
  { $project: { <campo1>: <incluir/excluir>, <campo2>: <incluir/excluir>, ... } }
])
```

Selección de campos con `$project`

Para obtener solo los nombres y edades de los empleados, se ejecutaría:

```
db.empleados.aggregate([
  { $project: { "nombre": 1, "edad": 1, "_id": 0 } }
])
```

Filtrado de documentos con `$match` :

```
db.<nombreColeccion>.aggregate([
  { $match: { <criterio> } }
])
```

Filtrado de documentos con `$match`

Para filtrar empleados del departamento de *Ventas*, se ejecutaría:

```
db.empleados.aggregate([
  { $match: { "departamento": "Ventas" } }
])
```

Podemos utilizar condiciones complejas con operadores de comparación y lógicos dentro de `$match` para construir consultas avanzadas.

Condiciones complejas con `$match`

Para filtrar empleados que son mayores de 30 años y pertenecen al departamento de *Ventas*, se ejecutaría:

```
db.empleados.aggregate([
  { $match: {
    $and: [
      { "edad": { $gt: 30 } },
      { "departamento": "Ventas" }
    ]
  } }
])
```

Con el Aggregation Framework es posible reliazar consultas con campos calculados. Esto **no se puede realizar** con `find()`.

Campos calculados con `$project`

Para calcular el año de nacimiento de los empleados a partir de su edad, se ejecutaría:

```
db.empleados.aggregate([
  { $project: {
    "nombre": 1,
    "edad": 1,
    "anio_nacimiento": { $subtract: [2024, "$edad"] },
    "_id": 0
  } }
])
```

Ordenación de resultados con `$sort`

```
db.<nombreColeccion>.aggregate([
  { $sort: { <campo1>: <orden1>, <campo2>: <orden2>, ... } }
])
```



Ordenación de resultados con \$sort

Para ordenar los empleados por edad de forma descendente, se ejecutaría:

```
db.empleados.aggregate([
  { $sort: { "edad": -1 } }
])
```

Limitar el número de resultados con `$limit` y `$skip`:

```
db.<nombreColeccion>.aggregate([
  { $limit: <número> }
])
```



Limitar resultados con \$limit

Para obtener solo los 5 empleados más jóvenes, se ejecutaría:

```
db.empleados.aggregate([
  { $sort: { "edad": 1 } },
  { $limit: 5 }
])
```



Paginación de resultados con \$skip y \$limit

Para obtener los empleados del segundo grupo de 5 empleados más jóvenes, se ejecutaría:

```
db.empleados.aggregate([
  { $sort: { "edad": 1 } },
  { $skip: 5 },
  { $limit: 5 }
])
```

Consultas de agrupación con \$group

El operador `$group` permite agrupar documentos por un campo específico y realizar operaciones de agregación como contar, sumar, promediar, etc. Su sintaxis es la siguiente:


```
db.<nombreColeccion>.aggregate([
  { $group: {
    _id: "$<campoAgrupacion>",
    <campoAgregado1>: { <operaciónAgregación1>: "$<campo1>" },
    <campoAgregado2>: { <operaciónAgregación2>: "$<campo2>" },
    // ...
  } }
])
```

Agrupación con \$group

Para contar el número de empleados por departamento, se ejecutaría:

```
db.empleados.aggregate([
  { $group: {
    _id: "$departamento",
    total_empleados: { $sum: 1 }
  } }
])
```

Se pueden combinar `$group` con `$project` para dar formato a los resultados de la agrupación.

Agrupación con \$group y proyección

Sumar el número de años de los empleados por departamento y mostrar el resultado con un formato específico, se ejecutaría:

```
db.empleados.aggregate([
  { $group: {
    _id: "$departamento",
    total_empleados: { $sum: 1 }
  } },
  { $project: {
    "departamento": "$_id",
    "total_empleados": 1,
    "_id": 0
  } }
])
```

En este ejemplo, se agrupan los empleados por departamento y se cuenta el número de empleados en cada uno. Luego, con `$project`, se da formato al resultado para mostrar el

nombre del departamento y el total de empleados, excluyendo el campo `_id` que se incluye por defecto en la agrupación.

Se pueden imponer condiciones de filtrado antes o después de la etapa de agrupación para obtener resultados más específicos. Por ejemplo, para contar el número de empleados por departamento solo para aquellos empleados mayores de 30 años, se ejecutaría:

Agrupación con \$group y filtrado

```
db.empleados.aggregate([
  { $match: { "edad": { $gt: 30 } } },
  { $group: {
    _id: "$departamento",
    total_empleados: { $sum: 1 }
  } },
  { $project: {
    "departamento": "$_id",
    "total_empleados": 1,
    "_id": 0
  } }
])
```

También se pueden utilizar operadores de agrupación más avanzados como `$avg` para calcular promedios, `$max` para obtener el valor máximo, `$min` para obtener el valor mínimo, entre otros.

Operadores de agrupación avanzados

Para calcular la edad promedio de los empleados por departamento, y mostrar sólo aquéllos departamentos con una edad promedio mayor a 30 años, se ejecutaría:

```

db.empleados.aggregate([
  { $group: {
    _id: "$departamento",
    edad_promedio: { $avg: "$edad" }
  } },
  { $project: {
    "departamento": "$_id",
    "edad_promedio": 1,
    "_id": 0
  } },
  { $match: { "edad_promedio": { $gt: 30 } } }
])

```

Como se puede observar podemos realizar consultas muy complejas utilizando el Aggregation Framework.

También se pueden realizar consultas que involucren operaciones de join entre colecciones utilizando el operador `$lookup`, y consultas que descompongan arrays en documentos individuales utilizando el operador `$unwind`, entre otras posibilidades avanzadas. Esto se verá posteriormente en la sección de relaciones entre colecciones.

Actualización de documentos en colecciones

Sobre los documentos almacenados en las colecciones, es posible realizar diversas operaciones de actualización para modificar su contenido. MongoDB proporciona comandos específicos para actualizar documentos de manera eficiente, permitiendo tanto modificaciones puntuales como actualizaciones masivas.

Insertar documentos

Para insertar documentos en una colección, se utilizan los comandos `insertOne` e `insertMany`. El comando `insertOne` permite añadir un único documento a una colección, mientras que `insertMany` se utiliza para insertar múltiples documentos en una sola operación. Su sintaxis es la siguiente:

```

db.<nombreColeccion>.insertOne({<documento>})
db.<nombreColeccion>.insertMany([<documento1>], [<documento2>], ...])

```



Insertar un documento

Para insertar un documento en la colección `empleados`, se ejecutaría:

```
db.empleados.insertOne({
  "nombre": "Carlos",
  "edad": 28,
  "departamento": "Ventas"
})
```

Insertar múltiples documentos

Para insertar varios documentos en la colección `empleados`, se ejecutaría:

```
db.empleados.insertMany([
  {
    "nombre": "Ana",
    "edad": 32,
    "departamento": "Marketing"
  },
  {
    "nombre": "Luis",
    "edad": 45,
    "departamento": "Finanzas"
  }
])
```

A diferencia de las bases de datos relacionales, en MongoDB no es necesario definir un esquema rígido para los documentos antes de insertarlos. Esto significa que cada documento puede tener una estructura diferente, aunque generalmente se recomienda mantener cierta consistencia en la estructura de los documentos dentro de una colección para facilitar su gestión y consulta.

Insertar documentos con estructura diferente

```

db.empleados.insertMany([
  {
    "nombre": "María",
    "edad": 30,
    "departamento": "Recursos Humanos",
    "idiomas": ["Español", "Francés"]
  },
  {
    "nombre": "Jorge",
    "edad": 40,
    "departamento": "Tecnología",
    "proyectos": [
      { "nombre": "Proyecto A", "duración": 6 },
      { "nombre": "Proyecto B", "duración": 12 }
    ]
  },
  {
    "nombre": "Sofía",
    "edad": 27,
    "departamento": "Administrativo",
  }
])

```

Modificar documentos

El **comando** `updateOne` modifica un único documento que coincida con el criterio especificado, mientras que el **comando** `updateMany` actualiza todos los documentos que cumplan con el criterio. Su sintaxis es la siguiente:

```

db.<nombreColeccion>.updateOne({<criterio>}, { $set: {<campos>} })
db.<nombreColeccion>.updateMany({<criterio>}, { $set: {<campos>} })

```



Actualizar un documento

Para actualizar la edad de un empleado llamado *Carlos* a 29 años, se ejecutaría:

```

db.empleados.updateOne(
  { "nombre": "Carlos" },
  { $set: { "edad": 29 } }
)

```

⌵ Actualizar múltiples documentos

Para actualizar el departamento de todos los empleados mayores de 40 años a *Senior*, se ejecutaría:

```
db.empleados.updateMany(  
  { "edad": { $gt: 40 } },  
  { $set: { "departamento": "Senior" } }  
)
```

En ambos comandos, el operador de actualización `$set` se utiliza para especificar los campos que se desean actualizar. Además de `$set`, existen otros operadores de actualización como `$inc` para incrementar valores numéricos, `$unset` para eliminar campos, `$push` para agregar elementos a un array, y `$pull` para eliminar elementos de un array que coincidan con una condición.

⌵ Actualizar lista

El siguiente ejemplo actualiza la lista de provincias de la *Comunidad Valenciana* eliminando la provincia *Murcia*.

```
db.comunidades.updateOne(  
  {"comunidad": 'Comunidad Valenciana'},  
  { $pull: {"provincias": "Murcia"} }  
)
```

⌵ Actualizar lista

El siguiente ejemplo actualiza la lista de idiomas del empleado *Carlos* añadiendo el *Polaco*.

```
db.empleados.updateOne(  
  { "nombre": "Carlos" },  
  { $push: { "idiomas": "Polaco" } }  
)
```

Es posible reemplazar un documento completo utilizando el **comando** `replaceOne`. Este comando reemplaza un documento que coincida con el criterio especificado por un nuevo documento completo, eliminando el documento original y creando uno nuevo con el contenido proporcionado.

```
db.<nombreColeccion>.replaceOne({<criterio>}, {<nuevoDocumento>})
```



Reemplazar un documento

Para reemplazar el documento de un empleado llamado *Ana* con un nuevo documento, se ejecutaría:

```
db.empleados.replaceOne(  
  { "nombre": "Ana" },  
  {  
    "nombre": "Ana García",  
    "edad": 33,  
    "departamento": "Marketing"  
  }  
)
```

Eliminar documentos

El **comando** `deleteOne` elimina un único documento que coincida con el criterio especificado, mientras que el **comando** `deleteMany` elimina todos los documentos que cumplan con el criterio. Su sintaxis es la siguiente:

```
db.<nombreColeccion>.deleteOne({<criterio>})  
db.<nombreColeccion>.deleteMany({<criterio>})
```



Eliminar un documento

Para eliminar un empleado llamado *Luis*, se ejecutaría:

```
db.empleados.deleteOne({ "nombre": "Luis" })
```



Eliminar múltiples documentos

Para eliminar todos los empleados del departamento de *Ventas*, se ejecutaría:

```
db.empleados.deleteMany({ "departamento": "Ventas" })
```

Uso básico de MongoDB Compass

Sección en construcción.

Modelado de datos en MongoDB

El modelado de datos en MongoDB se basa en la flexibilidad de los documentos BSON, lo que permite adaptar la estructura de la base de datos a las necesidades reales de la aplicación, a diferencia del modelo relacional tradicional. Esta flexibilidad facilita la optimización del rendimiento y la adecuación a los patrones de acceso más frecuentes. Tenemos dos estrategias principales para modelar las relaciones entre documentos:

- **Modelo embebido (embedding):** consiste en almacenar documentos relacionados dentro de un mismo documento principal. Esta estrategia es especialmente útil cuando los datos se consultan juntos habitualmente.
- **Modelo referenciado (referencing):** en este modelo, los documentos se relacionan mediante identificadores, de forma similar a las claves foráneas en bases de datos relacionales. Es adecuado para relaciones complejas o cuando los datos asociados pueden crecer mucho o ser compartidos.

A la hora de elegir entre estas dos estrategias, es importante considerar factores como la frecuencia de acceso a los datos, la necesidad de atomicidad en las operaciones, el tamaño de los documentos y la posibilidad de duplicación de información.

Relaciones 1–N, N–N, y 1–1 pueden ser modeladas utilizando tanto el modelo embebido como el modelo referenciado, dependiendo de las necesidades específicas de la aplicación y del patrón de acceso a los datos.

Como norma general, el modelo embebido suele ser más eficiente para relaciones 1-1 y 1-N, ya que permite acceder a toda la información relacionada en una sola consulta. Sin embargo, el modelo referenciado puede ser más adecuado para relaciones M-N o cuando los datos relacionados pueden crecer de manera significativa o ser compartidos entre múltiples documentos.

Relaciones 1-1

A continuación se muestran ejemplos de cómo modelar una relación 1-1 entre una persona y su pasaporte utilizando tanto embedding como referencing.

Suponemmos que tenemos una **persona** con los siguientes datos:


```
{
  "nombre": "Miguel",
  "apellidos": "Sánchez",
  "telefono": "555345623"
}
```

y una **documentación** para esa persona con los siguientes datos:

```
{
  "dni": "12341234T",
  "seguridad_social": "234298237PRQ"
}
```

Método *embedding* para relación 1-1

Para modelar una relación 1-1 entre una persona y su documentación utilizando *embedding*, se podría estructurar el documento de la siguiente manera:

```
{
  "nombre": "Miguel",
  "apellidos": "Sánchez",
  "telefono": "555345623",
  "documentacion": {
    "dni": "12341234T",
    "seguridad_social": "234298237PRQ"
  }
}
```

Método *referencing* para relación 1-1

Para modelar una relación 1-1 entre una persona y su documentación utilizando *referencing*, se podrían tener dos documentos separados:

```
// personas
{
  "nombre": "Miguel",
  "apellidos": "Sánchez",
  "telefono": "555345623",
  "documentacion_id": "1"
}
// documentacion
{
  "_id": "1",
  "dni": "12341234T",
  "seguridad_social": "234298237PRQ"
}
```

En este último caso, el documento de la persona contiene una referencia al documento de la documentación mediante el campo `documentacion_id`.

Relaciones 1-N

A continuación se muestran ejemplos de cómo modelar una relación 1-N entre una persona y sus vehículos utilizando tanto *embedding* como *referencing*.

Supongamos que tenemos una **persona** con los siguientes datos:

```
{
  "nombre": "Miguel",
  "apellidos": "Sánchez",
  "telefono": "555345623"
}
```

y una lista de **vehículos** con los siguientes datos:

```
[
  { "matricula": "1111AAA", "marca": "Toyota", "modelo": "Corolla" },
  { "matricula": "3344CCF", "marca": "Ford", "modelo": "Focus" }
]
```



Método *embedding* para relación 1-N

Para modelar una relación 1-N entre una persona y sus vehículos utilizando *embedding*, se podría estructurar el documento de la siguiente manera:

```
{
  "nombre": "Miguel",
  "apellidos": "Sánchez",
  "telefono": "555345623",
  "vehiculos": [
    { "matricula": "1111AAA", "marca": "Toyota", "modelo": "Corolla" },
    { "matricula": "3344CCF", "marca": "Ford", "modelo": "Focus" }
  ]
}
```

Para el caso referenciado, tenemos dos posibilidades de modelar la relación 1-N entre una persona y sus vehículos utilizando *referencing*. En la primera opción, el documento de la persona contiene una lista de referencias a los documentos de los vehículos. En la segunda opción, cada documento de vehículo contiene una referencia al documento de la persona.

Método *referencing* para relación 1-N

Modelo con referencias en la persona:

```
// personas
{
  "nombre": "Miguel",
  "apellidos": "Sánchez",
  "telefono": "555345623",
  "vehiculos": ["1", "2"]
}
// vehiculos
{ "_id": "1", "matricula": "1111AAA", "marca": "Toyota", "modelo": "Corolla" }
{ "_id": "2", "matricula": "3344CCF", "marca": "Ford", "modelo": "Focus" }
```

Método *referencing* para relación 1-N

Modelo con referencias en los vehículos:

```
// personas
{
  "_id": "1",
  "nombre": "Miguel",
  "apellidos": "Sánchez",
  "telefono": "555345623"
}
// vehiculos
{
  "matricula": "1111AAA",
  "marca": "Toyota",
  "modelo": "Corolla",
  "persona": "1"
}
{
  "matricula": "3344CCF",
  "marca": "Ford",
  "modelo": "Focus",
  "persona": "1"
}
```

Esto es equivalente a tener una clave foránea en el modelo relacional que apunta a la tabla de personas.

En una relación 1-N con el modelo referenciado, el guardar todas las referencias a los documentos relacionados en el documento principal puede resultar en un documento muy grande si la cantidad de documentos relacionados es considerable. En este caso, es recomendable utilizar el modelo referenciado con referencias en los documentos relacionados para evitar problemas de tamaño de documento y mejorar el rendimiento de las consultas.

También se puede optar por mantener tanto la lista de referencias en el documento principal como las referencias en los documentos relacionados, aunque esto puede llevar a una duplicación de información y a la necesidad de mantener la consistencia entre ambos conjuntos de referencias. Por eso, esta opción no es la más recomendable.

Migración de relaciones 1-N de un modelo relacional a MongoDB

En el supuesto que queramos migrar de una base de datos relacional a MongoDB y decidimos utilizar el **modelo embebido** para modelar una relación 1-N, hay que tener en cuenta que en muchas ocasiones los campos identificadores en el modelo relacional no forman parte de los documentos embebidos, ya que no son necesarios para identificar los documentos dentro del documento principal. Sin embargo, se pueden incluir si representan información relevante o si se desea mantener una referencia explícita a los identificadores originales de la base de datos relacional.

Los campos que actúan como claves foráneas en el modelo relacional se pueden excluir, si nos interesa, del documento embebido ya que aparecen dentro del documento principal.

Para los **modelos referenciados**, podemos mantener los campos identificadores como parte de los documentos referenciados para preservar la relación entre los documentos y facilitar la consulta y actualización de los datos relacionados. O bien, podemos optar por no incluir los campos identificadores en los documentos referenciados si consideramos que no son necesarios para la identificación de los documentos dentro de la colección de MongoDB, y en su lugar, utilizar el campo `_id` generado automáticamente por MongoDB como identificador único para cada documento. En este caso, las referencias entre los documentos se establecerían utilizando el campo `_id` en lugar de los campos identificadores originales del modelo relacional.

Relaciones M-N

A continuación se muestran ejemplos de cómo modelar una relación M-N entre libros y autores utilizando tanto embedding como referencin.

Supongamos que queremos modelar una relación M-N entre profesores y materias. Tenemos una colección de **profesores** con los siguientes datos:

```
[
  {
    "dni": "12345678A",
    "nombre": "Ana López",
    "tipoPlaza": "Titular"
  },
  {
    "dni": "23456789B",
    "nombre": "Bruno García",
    "tipoPlaza": "Asociado"
  },
  {
    "dni": "34567890C",
    "nombre": "Carla Sánchez",
    "tipoPlaza": "Catedrático"
  }
]
```

y otra de **materias** con los siguientes datos:

```
{ "nombre": "Álgebra Lineal", "creditos": "6", "curso": "1" },
{ "nombre": "Cálculo I", "creditos": "6", "curso": "1" },
{ "nombre": "Estadística", "creditos": "6", "curso": "2" },
{ "nombre": "Ecuaciones Diferenciales", "creditos": "6", "curso": "2" }
```

Utilizando la **estrategia de embedding**, podríamos almacenar la información de las materias dentro de cada documento de profesor, lo que permitiría acceder a toda la información relacionada con un profesor en una sola consulta.

En este caso para los **profesores** tenemos lo siguiente:

```
[
  {
    "dni": "12345678A",
    "nombre": "Ana López",
    "tipoPlaza": "Titular"
    "materias": [
      { "nombre": "Álgebra Lineal", "creditos": "6", "curso": "1" },
      { "nombre": "Cálculo I", "creditos": "6", "curso": "1" }
    ]
  },
  {
    "dni": "23456789B",
    "nombre": "Bruno García",
    "tipoPlaza": "Asociado"
    "materias": [
      { "nombre": "Estadística", "creditos": "6", "curso": "2" },
      { "nombre": "Ecuaciones Diferenciales", "creditos": "6", "curso": "2" }
    ]
  },
  {
    "dni": "34567890C",
    "nombre": "Carla Sánchez",
    "tipoPlaza": "Catedrático"
    "materias": [
      { "nombre": "Cálculo I", "creditos": "6", "curso": "1" },
      { "nombre": "Estadística", "creditos": "6", "curso": "2" },
      { "nombre": "Ecuaciones Diferenciales", "creditos": "6", "curso": "2" }
    ]
  }
]
```

De esta forma estamos privilegiando la eficiencia en las consultas sobre los profesores, ya que toda la información relacionada con cada profesor está contenida en un único documento. Por contra, las consultas sobre las materias serían menos eficientes, ya que tendríamos que recorrer todos los documentos de profesores para encontrar la información de una materia específica.

Una solución sería tener también una colección de **materias** con todos los profesores que imparten cada materia.

```
[
  {
    "nombre": "Álgebra Lineal",
    "creditos": "6",
    "curso": "1",
    "profesores": [
      { "dni": "12345678A", "nombre": "Ana López", "tipoPlaza": "Titular" }
    ]
  },
  {
    "nombre": "Cálculo I",
    "creditos": "6",
    "curso": "1",
    "profesores": [
      { "dni": "12345678A", "nombre": "Ana López", "tipoPlaza": "Titular" },
      { "dni": "34567890C", "nombre": "Carla Sánchez", "tipoPlaza": "Catedrático" }
    ]
  },
  {
    "nombre": "Estadística",
    "creditos": "6",
    "curso": "2",
    "profesores": [
      { "dni": "23456789B", "nombre": "Bruno García", "tipoPlaza": "Asociado" },
      { "dni": "34567890C", "nombre": "Carla Sánchez", "tipoPlaza": "Catedrático" }
    ]
  },
  {
    "nombre": "Ecuaciones Diferenciales",
    "creditos": "6",
    "curso": "2",
    "profesores": [
      { "dni": "23456789B", "nombre": "Bruno García", "tipoPlaza": "Asociado" },
      { "dni": "34567890C", "nombre": "Carla Sánchez", "tipoPlaza": "Catedrático" }
    ]
  }
]
```

Ahora tanto las consultas sobre profesores como las consultas sobre materias serían eficientes, ya que toda la información relacionada con cada profesor y cada materia está contenida en un único documento. Sin embargo, esto implica una enorme cantidad de información duplicada, lo que puede llevar a problemas de consistencia y a la necesidad de mantener la información actualizada en múltiples lugares.

Si utilizamos la **estrategia de referencing**, podríamos almacenar la información de las materias en una colección separada y referenciarla desde los documentos de los profesores mediante identificadores. De forma análoga, podríamos almacenar la información de los profesores en una

colección separada y referenciarla desde los documentos de las materias. A continuación se muestra un ejemplo de cómo modelar esta relación M-N utilizando *referencing*.

Para los documentos de **profesores**, tendríamos lo siguiente:

```
[
  {
    "_id": "100",
    "dni": "12345678A",
    "nombre": "Ana López",
    "tipoPlaza": "Titular",
    "materias": ["1", "2"]
  },
  {
    "_id": "101",
    "dni": "23456789B",
    "nombre": "Bruno García",
    "tipoPlaza": "Asociado",
    "materias": ["3", "4"]
  },
  {
    "_id": "102",
    "dni": "34567890C",
    "nombre": "Carla Sánchez",
    "tipoPlaza": "Catedrático",
    "materias": ["2", "3", "4"]
  }
]
```

Para los documentos de **materias**, tendríamos lo siguiente:


```
[
  {
    "_id": "1",
    "nombre": "Álgebra Lineal",
    "creditos": "6",
    "curso": "1",
    "profesores": ["100"]
  },
  {
    "_id": "2",
    "nombre": "Cálculo I",
    "creditos": "6",
    "curso": "1",
    "profesores": ["100", "102"]
  },
  {
    "_id": "3",
    "nombre": "Estadística",
    "creditos": "6",
    "curso": "2",
    "profesores": ["101", "102"]
  },
  {
    "_id": "4",
    "nombre": "Ecuaciones Diferenciales",
    "creditos": "6",
    "curso": "2",
    "profesores": ["101", "102"]
  }
]
```

Consultas multitaslas

La mayor parte de consultas multitabla utilizan el marco de agregación para realizar operaciones de join entre colecciones utilizando el operador `$lookup`, y consultas que descompongan arrays en documentos individuales utilizando el operador `$unwind`, entre otras posibilidades avanzadas. Cuando se utilizan documentos embebidos para modelar relaciones 1-N, se pueden realizar consultas utilizando el operador `$unwind` para descomponer los documentos embebidos en documentos individuales y luego aplicar filtros o proyecciones sobre los campos deseados. Cuando aplicamos el operador `$unwind` sobre un campo que contiene un array de documentos embebidos, MongoDB genera un nuevo documento para cada elemento del array, manteniendo los campos del documento principal. Esto permite realizar consultas específicas sobre los elementos embebidos como si fueran documentos independientes. El siguiente ejemplo ilustra el uso de `$unwind` para consultar documentos con documentos embebidos.

≡ Consulta multitabla (embedding)

En el siguiente ejemplo obtenemos el nombre del equipo y el nombre de cada base para los equipos de la división *Atlantic* teniendo en cuenta que los jugadores están embebidos dentro de los documentos de la colección `equipos`. El resultado debe estar ordenado por el nombre del equipo y el nombre del jugador de forma ascendente.

```
db.equipos.aggregate([
  { $match: { Division: "Atlantic" } },
  { $unwind: "$jugadores" },
  { $match: { "jugadores.Posicion": "G" } },
  {
    $project: {
      _id: 0,
      equipo: "$Nombre",
      jugador: "$jugadores.Nombre"
    }
  },
  {
    $sort: { equipo: 1, jugador: 1 }
  }
]);
```

A continuación se explica el resultado de cada etapa del pipeline:

- En la primera etapa (`$match`), se filtran los documentos de la colección `equipos` para obtener solo aquellos que pertenecen a la división *Atlantic*. En total son 6 equipos.
- En la segunda etapa (`$unwind`), se descompone el array de jugadores embebido en cada documento de equipo, generando un nuevo documento para cada jugador (incluyendo los datos del equipo al que pertenece). La variable `$jugadores` se utiliza para referirse a cada jugador individualmente en las etapas posteriores.
- En total 72 documentos correspondientes a los 72 jugadores de los 6 equipos.
- En la tercera etapa (`$match`), se filtran los documentos resultantes para obtener solo aquellos jugadores que ocupan la posición de base (G). En total 22 jugadores.
- En la cuarta etapa (`$project`), se proyectan solo los campos deseados, en este caso el nombre del equipo y el nombre del jugador, excluyendo el campo `_id`.
- Finalmente, en la última etapa (`$sort`), se ordenan los resultados por el nombre del equipo y el nombre del jugador de forma ascendente.

A continuación se muestra un ejemplo de cómo realizar una consulta multitabla utilizando el modelo referenciado con el operador `$lookup` para realizar un join entre las colecciones `equipos` y

Consulta multitabla (referencing)

En el siguiente ejemplo obtenemos el nombre del equipo y el nombre de cada base para los equipos de la división *Atlantic* teniendo en cuenta que los jugadores referencian al equipo al que pertenecen mediante el campo `equipo` . El resultado debe estar ordenado por el nombre del equipo y el nombre del jugador de forma ascendente.

```
db.jugadores.aggregate([
  {
    $lookup: {
      from: "equipos",
      localField: "equipo",
      foreignField: "_id",
      as: "equipoInfo"
    }
  },
  { $unwind: "$equipoInfo" },
  {
    $match: {
      Posicion: "G",
      "equipoInfo.Division": "Atlantic"
    }
  },
  {
    $project: {
      _id: 0,
      equipo: "$equipoInfo.Nombre",
      jugador: "$Nombre"
    }
  },
  {
    $sort: { equipo: 1, jugador: 1 }
  }
]);
```

A continuación se explica el resultado de cada etapa del pipeline:

- En la primera etapa (`$lookup`), se realiza un join entre la colección `jugadores` y la colección `equipos` , donde se especifica que el campo `equipo` de la colección `jugadores` se corresponde con el campo `_id` de la colección `equipos` . El resultado del join se almacena en un nuevo campo llamado `equipoInfo` , que contiene un array de documentos

correspondientes al equipo al que pertenece cada jugador. En total 432 documentos correspondientes a todos los jugadores que tenemos en nuestra base de datos.

- En la segunda etapa (`$unwind`), se descompone el array `equipoInfo` para obtener un documento individual para cada jugador con la información del equipo correspondiente (seguimos teniendo 432 documentos).
- En la tercera etapa (`$match`), se filtran los documentos resultantes para obtener solo aquellos jugadores que ocupan la posición de base (G) y pertenecen a equipos de la división *Atlantic*. En total 22 jugadores.
- En la cuarta etapa (`$project`), se proyectan solo los campos deseados, en este caso el nombre del equipo y el nombre del jugador, excluyendo el campo `_id`.
- Finalmente, en la última etapa (`$sort`), se ordenan los resultados por el nombre del equipo y el nombre del jugador de forma ascendente.

Los ejemplos vistos hasta ahora trabajaban con colecciones que mantenía una relación 1-N entre ellas. Para el caso de relaciones M-N, las consultas pueden ser más complejas, especialmente si se utiliza el modelo embebido, ya que puede haber duplicación de información y la necesidad de eliminar duplicados en los resultados. En el caso del modelo referenciado, las consultas pueden requerir múltiples etapas de `$lookup` para realizar joins entre las colecciones relacionadas.

A continuación vamos a ver una serie de ejemplos de consultas multitabla utilizando tanto el modelo embebido como el modelo referenciado para ilustrar cómo realizar consultas en ambos casos.

Retomamos el ejemplo de la facultad de Matemáticas con profesores y materias para ilustrar cómo realizar consultas multitabla en ambos casos. Vamos a considerar que inicialmente solo tenemos la colección de profesores con las materias embebidas.



Consulta multitabla (M-N embedding)

Necesito saber el nombre y la plaza de todos los profesores que imparten Estadística.

```

db.profesores.aggregate(
[
  { $unwind: "$materias" },
  { $match: { "materias.nombre": "Estadística" } },
  {
    $project: {
      _id: 0,
      nombre: "$nombre",
      tipoPlaza: "$tipoPlaza"
    }
  }
]);

```

Esta consulta es similar a las que hemos visto anteriormente.

Consulta multitabla (M-N embedding)

Necesito saber el nombre y los créditos de todas las materias impartidas por todos los profesores que son catedráticos (elimina duplicados).

```

db.profesores.aggregate(
[
  { $match: { "tipoPlaza": "Catedrático" } },
  { $unwind: "$materias" },
  {
    $project: {
      _id: 0,
      nombre: "$materias.nombre",
      creditos: "$materias.creditos"
    }
  },
  {
    $group: { _id: "$nombre" }
  },
  {
    $project: {
      _id: 0,
      nombre: "$_id",
      creditos: 1
    }
  }
]);

```

Esta consulta es bastante más compleja que las anteriores, ya que incluye una etapa de agrupación para eliminar los duplicados de materias impartidas por varios catedráticos.

La consulta consiste en las siguientes etapas:

- En la primera etapa (`$match`), se filtran los documentos de la colección `profesores` para obtener solo aquellos que tienen el tipo de plaza "Catedrático".
- En la segunda etapa (`$unwind`), se descompone el array de materias embebido en cada documento de profesor, generando un nuevo documento para cada materia impartida por los catedráticos.
- En la tercera etapa (`$project`), se proyectan solo los campos deseados, en este caso el nombre de la materia y los créditos, excluyendo el campo `_id`.
- En la cuarta etapa (`$group`), se agrupan los documentos por el nombre de la materia para eliminar los duplicados, utilizando el operador `$first` para conservar los créditos de la primera aparición de cada materia.
- Finalmente, en la última etapa (`$project`), se proyectan nuevamente los campos deseados, en este caso el nombre de la materia y los créditos, excluyendo el campo `_id`.

Evidentemente, si hubiéramos tenido embebidas también las materias dentro de los documentos de los profesores, la consulta sería mucho más sencilla, ya que no tendríamos que preocuparnos por eliminar duplicados. La consulta sería similar a la primera consulta que hemos visto.

Consulta multitabla (M-N embedding)

Necesito saber el nombre y los créditos de todas las materias impartidas por todos los profesores que son catedráticos.

```
db.profesores.aggregate(  
  [  
    { $match: { "tipoPlaza": "Catedrático" } },  
    { $unwind: "$materias" },  
    {  
      $project: {  
        _id: 0,  
        nombre: "$materias.nombre",  
        creditos: "$materias.creditos"  
      }  
    }  
  ]  
);
```

Con el modelo referenciado, la consulta sería similar a la consulta multitabla que hemos visto anteriormente utilizando `$lookup` para realizar el join entre las colecciones de profesores y materias. Si suponemos que tenemos una colección de profesores con referencias a las materias que imparten y

una colección de materias con referencias a los profesores que las imparten. Con este supuesto realizaremos un par de consultas para ilustrar cómo realizar consultas multitabla en este caso.

Consulta multitabla (M-N referencing)

Necesito saber el nombre y los créditos de todas las materias impartidas por todos los profesores que son catedráticos.

```
db.profesores.aggregate([
  { $match: { tipoPlaza: "Catedrático" } },
  { $unwind: "$materias" },
  {
    $lookup: {
      from: "materias",
      localField: "materias",
      foreignField: "_id",
      as: "materiaInfo"
    }
  },
  { $unwind: "$materiaInfo" },
  {
    $project: {
      _id: 0,
      materia: "$materiaInfo.nombre",
      credits: "$materiaInfo.creditos"
    }
  }
]);
```

Las etapas de esta consulta son las siguientes:

- En la primera etapa (`$match`), se filtran los documentos de la colección `profesores` para obtener solo aquellos que tienen el tipo de plaza "Catedrático".
- En la segunda etapa (`$unwind`), se descompone el array de materias referenciadas en cada documento de profesor, generando un nuevo documento para cada materia impartida por los catedráticos.
- En la tercera etapa (`$lookup`), se realiza un join entre la colección `profesores` y la colección `materias`, donde se especifica que el campo `materias` de la colección `profesores` se corresponde con el campo `_id` de la colección `materias`. El resultado del join se almacena en un nuevo campo llamado `materiaInfo`, que contiene un array de documentos correspondientes a la materia impartida por cada profesor.
- En la cuarta etapa (`$unwind`), se descompone el array `materiaInfo` para obtener un documento individual para cada materia impartida por los catedráticos.

- En la quinta etapa (`$project`), se proyectan solo los campos deseados, en este caso el nombre de la materia y los créditos, excluyendo el campo `_id` .

Consulta multitabla (M-N referencing)

Necesito saber el nombre y la plaza de todos los profesores que imparten *Estadística* o *Cálculo I*.

```
db.materias.aggregate([
  { $match: { nombre: { $in: ["Estadística", "Cálculo I"] } } },
  { $unwind: "$profesores" },
  {
    $lookup: {
      from: "profesores",
      localField: "profesores",
      foreignField: "_id",
      as: "profesorInfo"
    }
  },
  { $unwind: "$profesorInfo" },
  {
    $group: {
      _id: "$profesorInfo._id",
      nombre: { $first: "$profesorInfo.nombre" },
      tipoPlaza: { $first: "$profesorInfo.tipoPlaza" }
    }
  },
  { $project: { _id: 0, nombre: 1, tipoPlaza: 1 } }
]);
```

Consulta similar a la anterior salvo que ahora es necesario hacer un `$group` para evitar duplicados cuando un profesor imparte ambas materias.

En la consulta anterior, partimos de la colección `materias` para obtener los profesores que imparten las materias de interés. Sin embargo, también podríamos partir de la colección `profesores` y realizar un `$lookup` para obtener la información de las materias que imparten cada profesor, y luego aplicar un filtro para obtener solo aquellos profesores que imparten las materias de interés. Esto nos permitiría evitar el uso del operador `$group` para eliminar duplicados, ya que estaríamos obteniendo un documento por profesor desde el principio.

≡ Consulta multitabla (M-N referencing)

Necesito saber el nombre y la plaza de todos los profesores que imparten *Estadística* o *Cálculo I*.

```
db.profesores.aggregate([
  {
    $lookup: {
      from: "materias",
      localField: "materias",
      foreignField: "_id",
      as: "materiasInfo"
    }
  },
  { $match: { "materiasInfo.nombre": { $in: ["Estadística", "Cálculo I"] } } },
  { $project: { _id: 0, nombre: 1, tipoPlaza: 1 } }
]);
```

Por último, vamos a ver un ejemplo de consulta multitabla más compleja utilizando el modelo referenciado, donde se utilizan dos etapas de `$lookup` para obtener información de ambas colecciones relacionadas.

≡ Consulta multitabla (M-N referencing, 2 \$lookup)

Para cada profesor, necesito una lista de elementos con: **nombre de materia** y **lista de nombres de los profesores** que imparten esa misma materia.

```

db.profesores.aggregate(
[
  {
    $lookup: {
      from: "materias",
      localField: "materias",
      foreignField: "_id",
      as: "materiasInfo"
    }
  },
  { $unwind: "$materiasInfo" },
  {
    $lookup: {
      from: "profesores",
      let: { materiaId: "$materiasInfo._id" },
      pipeline: [
        {
          $match: {
            $expr: { $in: ["$$materiaId", "$materias"] }
          }
        },
        { $project: { _id: 0, nombre: 1 } }
      ],
      as: "profesoresMismaMateria"
    }
  },
  {
    $project: {
      nombre: 1,
      tipoPlaza: 1,
      materia: "$materiasInfo.nombre",
      profesores: {
        $map: {
          input: "$profesoresMismaMateria",
          as: "p",
          in: "$$p.nombre"
        }
      }
    }
  },
  {
    $group: {
      _id: "$_id",
      nombre: { $first: "$nombre" },
      tipoPlaza: { $first: "$tipoPlaza" },
      materias: { $push: { materia: "$materia", profesores: "$profesores" } }
    }
  },
  { $project: { _id: 0, nombre: 1, tipoPlaza: 1, materias: 1 } }
]);

```

En este caso se usan 2 `$lookup` : el primero para obtener la información de las materias, y el segundo para obtener los profesores que imparten cada materia.

Otras aspectos importantes de MongoDB

A continuación se presentan otros aspectos importantes de MongoDB aunque se abordarán de forma muy superficial, ya que cada uno de ellos merece un tratamiento más detallado.

Índices

Los índices en MongoDB son estructuras de datos que mejoran la velocidad de las operaciones de consulta en una colección. Al crear un índice en uno o más campos de una colección, MongoDB puede buscar documentos de manera más eficiente, lo que reduce el tiempo de respuesta de las consultas. Los índices funcionan de manera similar a los índices en una base de datos relacional, permitiendo a MongoDB localizar rápidamente los documentos que coinciden con los criterios de búsqueda. Para crear un índice en MongoDB, se utiliza el método `createIndex()` , que acepta un objeto que especifica los campos a indexar y el tipo de índice (ascendente o descendente). La sintaxis es la siguiente:

```
db.<nombreColeccion>.createIndex({<campo1>: <tipo1>, <campo2>: <tipo2>, ...})
```



Crear índice

Para crear un índice ascendente en el campo "nombre" de la colección `empleados` , se ejecutaría:

```
db.empleados.createIndex({ "nombre": 1 })
```

Para crear un índice descendente en el campo "edad" de la colección `empleados` , se ejecutaría:

```
db.empleados.createIndex({ "edad": -1 })
```

Usuarios y roles

MongoDB proporciona un sistema de autenticación y autorización basado en usuarios y roles para controlar el acceso a las bases de datos y las operaciones que pueden realizar los usuarios. Los usuarios se crean con un nombre de usuario, una contraseña y una lista de roles que definen sus permisos. Los roles pueden ser predefinidos por MongoDB o personalizados según las necesidades

de la aplicación. Para crear un usuario en MongoDB, se utiliza el método `createUser()`, que acepta un objeto con la información del usuario y sus roles. La sintaxis es la siguiente:

```
db.createUser({
  user: "<nombreUsuario>",
  pwd: "<contraseña>",
  roles: [
    { role: "<rol1>", db: "<baseDatos1>" },
    { role: "<rol2>", db: "<baseDatos2>" },
    // ...
  ]
})
```

Crear usuario

Para crear un usuario llamado "admin" con la contraseña "admin123" y el rol de administrador en la base de datos "empresa", se ejecutaría:

```
db.createUser({
  user: "admin",
  pwd: "admin123",
  roles: [
    { role: "dbAdmin", db: "empresa" },
    { role: "readWrite", db: "empresa" }
  ]
})
```

En este ejemplo, el usuario "admin" tiene permisos de administración de la base de datos "empresa" y también puede leer y escribir datos en esa base de datos.

Para eliminar un usuario en MongoDB, se utiliza el método `dropUser()`, que acepta el nombre del usuario a eliminar. La sintaxis es la siguiente:

```
db.dropUser("<nombreUsuario>")
```

Eliminar usuario

Para eliminar el usuario "admin", se ejecutaría:

```
db.dropUser("admin")
```

Scripts de MongoDB

MongoDB permite ejecutar scripts escritos en JavaScript para realizar operaciones complejas o automatizar tareas. Estos scripts pueden contener una serie de comandos y operaciones que se ejecutan secuencialmente, lo que facilita la gestión y manipulación de los datos en la base de datos. Para ejecutar un script en MongoDB, se puede utilizar el método `load()`, que carga y ejecuta un archivo JavaScript desde el sistema de archivos. La sintaxis es la siguiente:

```
load("<rutaArchivo.js>")
```

Ejecutar script

Para ejecutar un script llamado "actualizar_empleados.js" ubicado en la carpeta "scripts", se ejecutaría:

```
load("scripts/actualizar_empleados.js")
```